UNIVERSIDADE PAULISTA – UNIP
Projeto Integrado Multidisciplinar
Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

JARBAS MOISES CAMPOS CAMARGO DO PRADO – T235FD4

LEANDRO DA SILVA CARVALHO JUNIOR – R4614B0

MARIA EDUARDA NAVA – R1932G1

RAFAELA AMBROSIO DE OLIVEIRA – R6620H5

SAFIRA ALVES BARROS – F360FE8

VITORIA DOS SANTOS AMORIM – H66150

PROJETO INTEGRADO MULTIDISCIPLINAR IV: SOLUÇÃO TECNOLÓGICA CORPORATIVA INTEGRADA PARA A TRANSFORMAÇÃO DIGITAL DA INSTITUIÇÃO LUMINA

Santana de Parnaíba

2026

JARBAS MOISES CAMPOS CAMARGO DO PRADO
LEANDRO DA SILVA CARVALHO JUNIOR
MARIA EDUARDA NAVA
RAFAELA AMBROSIO DE OLIVEIRA
SAFIRA ALVES BARROS
VITORIA DOS SANTOS AMORIM

PROJETO INTEGRADO MULTIDISCIPLINAR IV: SOLUÇÃO TECNOLÓGICA CORPORATIVA INTEGRADA PARA A TRANSFORMAÇÃO DIGITAL DA INSTITUIÇÃO LUMINA

Projeto Integrado Multidisciplinar para obtenção do título de tecnólogo em Análise e Desenvolvimento de Sistemas, apresentado à Universidade Paulista — UNIP.

Orientador: Tarcisio de Souza Peres

Santana de Parnaíba
2026

RESUMO

O presente trabalho apresenta o desenvolvimento e a implantação de uma solução tecnológica corporativa integrada para a instituição de ensino Lumina, organização simulada estudada no Projeto Integrado Multidisciplinar anterior. Dando continuidade ao levantamento então realizado, o projeto avançou da fase de análise para a construção efetiva do sistema, contemplando aplicação web, aplicativo móvel, banco de dados corporativo, arquitetura de software em camadas e plano de infraestrutura em nuvem. A aplicação web foi desenvolvida em C# sobre a plataforma .NET, expondo uma interface de programação de aplicações no padrão REST protegida por autenticação baseada em JSON Web Token e controle de acesso por perfis. O aplicativo móvel foi construído em React Native com o ecossistema Expo, consumindo a mesma interface de programação e compartilhando a base de usuários, o que assegura consistência entre as plataformas. A arquitetura foi reorganizada em quatro camadas — Domínio, Infraestrutura, Aplicação e Apresentação — com dependências explícitas entre os projetos, atendendo aos princípios de orientação a objetos e modularização. O banco de dados relacional, implementado em Microsoft SQL Server, foi normalizado até a terceira forma normal e teve as regras de negócio transpostas para procedimentos armazenados e gatilhos, de modo que as restrições passassem a valer independentemente do cliente que acessa os dados. A solução foi conteinerizada com Docker e implantada em nuvem, com a aplicação no Render e o banco no Azure SQL Database, sob integração contínua por GitHub Actions e entrega contínua a cada envio de código. Como diferencial, desenvolveu-se o Farol de Evasão, que cruza desempenho, frequência de acesso, leitura de conteúdos e atividades pendentes para sinalizar estudantes em risco de abandono, com justificativa escrita para cada alerta. O gerenciamento seguiu a metodologia ágil Scrum, com quadro Kanban na plataforma Trello e ciclos quinzenais. Em atendimento aos princípios de responsabilidade social, o sistema incorpora recursos de acessibilidade nas duas plataformas, incluindo alto contraste, escalas de texto e glossário em Língua Brasileira de Sinais, além de um assistente virtual que reduz barreiras de uso. Os resultados demonstram que a integração entre aplicações web e móveis, apoiada por arquitetura modular e banco de dados consistente, constitui caminho viável para a modernização tecnológica de instituições de ensino, promovendo eficiência operacional, inclusão e sustentabilidade pela redução do uso de papel.

Palavras-chave: Transformação Digital. Desenvolvimento Web. Desenvolvimento Mobile. Arquitetura de Software. Banco de Dados. Acessibilidade.

ABSTRACT

This paper presents the development and deployment of an integrated corporate technological solution for Lumina, a simulated educational institution studied in the previous Multidisciplinary Integrated Project. Building upon the analysis carried out at that stage, the project moved from assessment to the effective construction of the system, comprising a web application, a mobile application, a corporate database, a layered software architecture and a cloud infrastructure plan. The web application was developed in C# on the .NET platform, exposing a REST application programming interface protected by JSON Web Token authentication and role-based access control. The mobile application was built with React Native and the Expo ecosystem, consuming the same interface and sharing the user base, which ensures consistency across platforms. The architecture was reorganised into four layers — Domain, Infrastructure, Application and Presentation — with explicit dependencies between projects, meeting object-oriented and modularisation principles. The relational database, implemented in Microsoft SQL Server, was normalised to the third normal form and had its business rules transposed into stored procedures and triggers, so that constraints apply regardless of the client accessing the data. The solution was containerised with Docker and deployed to the cloud, with the application on Render and the database on Azure SQL Database, under continuous integration through GitHub Actions and continuous delivery on every code push. As a distinguishing feature, a Dropout Early-Warning module was developed, combining performance, access frequency, content reading and pending assignments to flag students at risk of dropping out, with a written justification for each alert. Project management followed the Scrum agile methodology, using a Kanban board on the Trello platform with fortnightly cycles. In accordance with social responsibility principles, the system incorporates accessibility resources on both platforms, including high contrast, text scaling and a Brazilian Sign Language glossary, as well as a virtual assistant that lowers usage barriers. The results show that integrating web and mobile applications, supported by a modular architecture and a consistent database, is a viable path for the technological modernisation of educational institutions, promoting operational efficiency, inclusion and sustainability through reduced paper consumption.

Keywords: Digital Transformation. Web Development. Mobile Development. Software Architecture. Database. Accessibility.

SUMÁRIO

1 INTRODUÇÃO 7

2 CARACTERIZAÇÃO DA ORGANIZAÇÃO 9

2.1 Segmento de atuação 9

2.2 Produtos e serviços 9

2.3 Estrutura organizacional 9

2.4 Principais processos 9

2.5 Problemas identificados 10

2.6 Oportunidades de melhoria por meio da tecnologia 10

3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA 11

3.1 Problema a ser resolvido 11

3.2 Objetivos da solução 11

3.3 Público-alvo 11

3.4 Proposta de valor 11

3.5 Benefícios esperados 12

3.6 Diferenciais competitivos 12

3.7 Modelo de negócio e viabilidade 12

4 RESPONSABILIDADE SOCIAL E DIVERSIDADE 13

4.1 Combate à discriminação 13

4.2 Valorização da diversidade 13

4.3 Recursos tecnológicos inclusivos 13

4.4 Ações voltadas à acessibilidade 14

4.5 Tecnologia para a promoção da cidadania 14

4.6 Farol de Evasão 14

5 DESENVOLVIMENTO DA SOLUÇÃO WEB 16

5.1 Arquitetura da aplicação 16

5.2 Interface de programação REST 17

5.3 Autenticação 17
5.4 Segurança 18
5.5 Estrutura administrativa 18
5.6 Assistente virtual 19
5.7 Integração entre módulos 20
5.8 Correções decorrentes de teste em dispositivo real 20
6 DESENVOLVIMENTO DA SOLUÇÃO MOBILE 21
6.1 Tecnologia adotada e justificativa 21
6.2 Telas principais 21
6.3 Fluxo de navegação 24
6.4 Autenticação e armazenamento seguro 24
6.5 Sincronização de dados 24
6.6 Integração com a interface de programação 24
6.7 Acessibilidade no aplicativo 25
6.8 Verificação 26
7 ARQUITETURA DE SOFTWARE 27
7.1 Situação inicial e motivação 27
7.2 Organização em camadas 27
7.3 Orientação a objetos 28
7.4 Modularização 28
7.5 Verificação da refatoração 28
8 PROJETO DO BANCO DE DADOS 29
8.1 Justificativa do modelo relacional 29
8.2 Modelo conceitual 29
8.3 Modelo lógico 30
8.4 Modelo físico 30
8.5 Procedimentos armazenados 31

8.6 Gatilhos 32
8.7 Verificação 32
8.8 Script completo 32
9 INFRAESTRUTURA EM NUVEM E DEVOPS 33
9.1 Arquitetura em nuvem 33
9.2 Serviços utilizados 33
9.3 Contêineres 33
9.4 Pipeline de integração e entrega contínuas 34
9.5 Monitoramento 34
9.6 Escalabilidade 35
9.7 Segurança da infraestrutura 35
9.8 Verificação do ambiente 35
10 GERENCIAMENTO ÁGIL DO PROJETO 36
10.1 Metodologia adotada 36
10.2 Papéis 36
10.3 Product Backlog 36
10.4 Sprint Backlog 37
10.5 Cronograma 37
10.6 Definição de pronto 38
11 CONCLUSÃO 39
REFERÊNCIAS 41

LISTA DE FIGURAS

Nenhuma entrada de índice de ilustrações foi encontrada.

1 INTRODUÇÃO

A transformação digital deixou de ser diferencial competitivo para tornar-se condição de permanência das organizações. No setor educacional, essa mudança assume contornos particulares: além de ganhos administrativos, a tecnologia altera a forma como estudantes acessam conteúdo, como docentes acompanham turmas e como instituições comprovam resultados pedagógicos. Nesse cenário, soluções que integrem processos administrativos e acadêmicos em um único ambiente digital tornam-se instrumentos de gestão e, simultaneamente, de inclusão.

O presente Projeto Integrado Multidisciplinar dá continuidade ao trabalho desenvolvido no PIM III, no qual foi realizado o levantamento de requisitos, a modelagem e a prototipação do Sistema Acadêmico Colaborativo para a instituição simulada Lumina. Enquanto aquela etapa concentrou-se na análise e no planejamento, esta avança para a construção efetiva da solução, contemplando o desenvolvimento das aplicações web e móvel, a reestruturação da arquitetura de software, a implementação completa do banco de dados corporativo e a definição da infraestrutura de implantação.

O objetivo geral deste trabalho é desenvolver e implantar uma solução tecnológica corporativa integrada capaz de apoiar a transformação digital da instituição Lumina, considerando aspectos de inovação, inclusão, desempenho, escalabilidade, segurança e qualidade. Como objetivos específicos, o projeto propõe-se a: construir uma aplicação web com interface de programação no padrão REST e autenticação segura; desenvolver um aplicativo móvel integrado a essa mesma interface; reorganizar a arquitetura em camadas com responsabilidades bem delimitadas; projetar e implementar um banco de dados relacional otimizado, com regras de negócio garantidas pelo próprio gerenciador; elaborar um plano de infraestrutura em nuvem apoiado em conteinerização e automação de implantação; e conduzir o desenvolvimento por meio de metodologia ágil.

Quanto à metodologia, o trabalho caracteriza-se como pesquisa aplicada de natureza exploratória, conduzida na forma de desenvolvimento experimental. A fundamentação teórica apoiou-se em obras de referência das áreas de engenharia de software, arquitetura de sistemas e bancos de dados, com destaque para Sommerville (2019), Pressman e Maxim (2016), Martin (2019) e Elmasri e Navathe (2018). A condução seguiu o framework Scrum, com ciclos quinzenais, quadro Kanban para acompanhamento das tarefas e reuniões periódicas de alinhamento. Cada funcionalidade implementada foi submetida a verificação prática antes de

ser considerada concluída, procedimento cujos resultados são apresentados ao longo dos capítulos.

A instituição objeto do estudo, Lumina, é uma organização de ensino simulada criada para o desenvolvimento acadêmico deste projeto. Trata-se de uma escola de porte médio que enfrenta desafios recorrentes no setor: dispersão de informações entre planilhas e documentos físicos, comunicação fragmentada entre os públicos internos e ausência de indicadores consolidados para apoiar decisões pedagógicas. Tais características, embora reproduzidas em ambiente controlado, refletem dificuldades concretas observadas em instituições reais, o que confere pertinência à solução proposta.

O documento está organizado em onze seções. Após esta introdução, a segunda seção caracteriza a organização estudada. A terceira apresenta o planejamento da solução sob a perspectiva do empreendedorismo em tecnologia da informação. A quarta discute responsabilidade social, diversidade e acessibilidade. Da quinta à nona seção são detalhados, respectivamente, o desenvolvimento web, o desenvolvimento móvel, a arquitetura de software, o projeto do banco de dados e a infraestrutura em nuvem. A décima seção descreve o gerenciamento ágil adotado, e a décima primeira apresenta as conclusões do trabalho.

2 CARACTERIZAÇÃO DA ORGANIZAÇÃO

2.1 Segmento de atuação

A Lumina é uma instituição de ensino privada de porte médio, situada no município de Santana de Parnaíba, no estado de São Paulo. Atua no segmento de educação básica, com oferta de Ensino Fundamental II e Ensino Médio, atendendo aproximadamente 800 estudantes distribuídos em dois períodos. O quadro funcional é composto por cerca de 45 docentes e 12 colaboradores técnico-administrativos.

Instituições desse porte ocupam posição peculiar no segmento privado: são grandes o bastante para que processos manuais se tornem gargalo, mas não dispõem de orçamento comparável ao de grandes redes para adquirir sistemas proprietários, cujo licenciamento costuma ser cobrado por aluno matriculado.

2.2 Produtos e serviços

A instituição oferece ensino regular presencial do Fundamental II e do Ensino Médio, em turmas de até 40 estudantes; acompanhamento pedagógico, com avaliações periódicas e comunicação de desempenho às famílias; material didático digital produzido pelo próprio corpo docente; e atendimento administrativo de matrículas, rematrículas e emissão de documentos escolares.

2.3 Estrutura organizacional

A estrutura organiza-se em três níveis. A **direção** responde pelas decisões estratégicas e pelo orçamento. A **coordenação pedagógica** acompanha o plano de ensino, orienta os docentes e monitora indicadores de aprendizagem. A **secretaria acadêmica** executa os processos administrativos e o atendimento de primeiro nível aos usuários dos sistemas internos. Sob a coordenação atua o corpo docente, responsável por aulas, material, avaliações e lançamento de notas; o corpo discente é o público final dos serviços.

Essa divisão relaciona-se diretamente com o controle de acesso adotado no sistema, apresentado na seção 5, no qual os privilégios foram segmentados nos perfis Secretaria, Professor e Aluno.

2.4 Principais processos

Quatro processos são centrais. A **matrícula e alocação** era conduzida em planilhas, sem validação automática do limite de vagas. A **gestão de conteúdo pedagógico** distribuía material por meios heterogêneos — mensageiros, correio eletrônico e cópias impressas —, sem

repositório central. A **avaliação** aplicava provas em papel, com correção manual e transcrição para planilhas, levando até duas semanas entre a aplicação e a divulgação. O **acompanhamento gerencial** dependia de levantamentos produzidos manualmente a partir das planilhas dos docentes.

2.5 Problemas identificados

A análise conduzida no PIM III identificou seis problemas, aqui retomados por constituírem a justificativa da solução. A **dispersão da informação** entre planilhas, impressos e mensagens pessoais elimina a fonte única de consulta e produz divergência entre registros. A **comunicação fragmentada** decorre da ausência de canal institucional único, gerando perda de prazos e retrabalho. A **ausência de indicadores** consolidados impede identificar tempestivamente estudantes com baixo rendimento, restringindo a atuação pedagógica ao caráter corretivo. A **sobrecarga administrativa** concentra na secretaria tarefas repetitivas — sobretudo recuperação de senha e dúvidas operacionais — que consomem tempo qualificado sem agregar valor. O **consumo de papel** decorre da impressão recorrente de avaliações e comunicados. As **barreiras de acessibilidade** impedem o uso autônomo dos sistemas por pessoas com deficiência visual ou auditiva.

2.6 Oportunidades de melhoria por meio da tecnologia

Do diagnóstico decorrem as oportunidades que orientaram as decisões técnicas descritas nos capítulos seguintes:

Problema identificado	**Oportunidade tecnológica**	**Seção correspondente**
Dispersão da informação	Banco de dados relacional único, normalizado e íntegro	8
Comunicação fragmentada	Plataforma web e móvel com base de dados compartilhada	5 e 6
Ausência de indicadores	Painéis com indicadores e classificação automática de desempenho	5
Sobrecarga administrativa	Assistente virtual para dúvidas recorrentes e autoatendimento	5
Consumo de papel	Digitalização de avaliações e materiais didáticos	5
Barreiras de acessibilidade	Recursos de acessibilidade nas duas plataformas	4

A solução não pretende substituir a atuação humana, mas eliminar o trabalho repetitivo que hoje a consome.

3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA

(Disciplina: Empreendedorismo em Tecnologia da Informação)

3.1 Problema a ser resolvido

O problema central: **a instituição Lumina não dispõe de ambiente digital unificado que integre os processos acadêmicos e administrativos, o que provoca dispersão de informações, retrabalho e ausência de indicadores para a decisão pedagógica.** Trata-se de problema de integração, e não de falta de ferramentas: planilhas e mensageiros resolvem tarefas isoladas, mas os dados não circulam entre eles.

3.2 Objetivos da solução

Objetivo geral: disponibilizar uma plataforma acadêmica integrada, acessível por navegador e por dispositivo móvel, que centralize o cadastro de usuários, a gestão de turmas e conteúdos, a aplicação de avaliações e a apuração de indicadores de desempenho.

Objetivos específicos: centralizar os dados acadêmicos em base única; divulgar resultados de avaliações imediatamente, por correção automática; identificar antecipadamente estudantes em risco; reduzir atendimentos da secretaria por autoatendimento; assegurar o uso autônomo por pessoas com deficiência; e reduzir o consumo de papel.

3.3 Público-alvo

A solução destina-se a três públicos internos, representados pelas personas definidas no PIM III: os **estudantes** (cerca de 800), pela persona Lucas, que acessa predominantemente pelo celular — o que justificou o aplicativo da seção 6 —; os **docentes** (cerca de 45), por Mariana, que precisa publicar materiais com agilidade e identificar estudantes com dificuldade; e a **equipe administrativa** (cerca de 12), por Carla, que cadastra usuários, efetiva matrículas e emite relatórios.

3.4 Proposta de valor

A proposta articula-se em quatro elementos: **unificação**, pois uma única base alimenta a aplicação web e o aplicativo; **imediatismo**, pois a correção automática converte um processo de até duas semanas em resultado instantâneo; **visibilidade**, pois indicadores e o Farol de Evasão deslocam a atuação da coordenação do corretivo para o preventivo; e **inclusão**, que estende a modernização aos usuários habitualmente preteridos.

3.5 Benefícios esperados

Dimensão	Benefício	Indicador
Operacional	Divulgação de resultados mais rápida	De 14 dias para imediata
Operacional	Menos atendimentos de suporte	Chamados de recuperação de senha
Pedagógica	Risco de evasão detectado cedo	Estudantes sinalizados pelo Farol de Evasão
Pedagógica	Mais acesso a materiais	Registros de leitura
Ambiental	Menos papel	Impressões de avaliações
Social	Uso autônomo por pessoa com deficiência	Recursos implementados

Cabe ressalva metodológica: por tratar-se de instituição simulada, os indicadores acima são parâmetros propostos para aferição em implantação real, não resultados mensurados.

3.6 Diferenciais competitivos

Cinco diferenciais distinguem a solução das alternativas de mercado. O principal é o **Farol de Evasão**, descrito na seção 4.6, que sinaliza estudantes em risco de abandono a partir de dados que o próprio sistema já registra — inclusive o tempo dedicado à leitura de conteúdos —, com justificativa escrita para cada alerta. Não há **licenciamento por usuário**: a solução usa apenas tecnologias sem custo de licença, e o custo de operação independe do porte (seção 3.7). Há **autonomia em relação a serviços externos**: o assistente virtual opera sobre base própria, sem custo por requisição e sem enviar dados de estudantes para fora da instituição. A **acessibilidade é nativa**, presente nas duas plataformas. E a **portabilidade** da conteinerização permite implantar em servidor próprio ou em qualquer provedor de nuvem.

3.7 Modelo de negócio e viabilidade

Para a Lumina, a solução é **ativo próprio**, cujo retorno se mede pela redução de trabalho manual e pelos indicadores da seção 3.5. Como desdobramento, poderia ser ofertada a outras instituições por **assinatura de valor fixo por instituição**, e não por estudante.

A viabilidade foi analisada em três dimensões. A **técnica** está demonstrada pela própria entrega: o sistema encontra-se implantado e em operação, conforme a seção 9. A **operacional** decorre do modelo de Plataforma como Serviço, que transfere ao provedor a manutenção de servidores, dispensando equipe dedicada. A **econômica** apoia-se no custo previsto para a operação real, que não cresce com o número de matrículas:

Componente	Serviço	Custo mensal
Aplicação, sem hibernação	Render, plano Starter	US$ 7,00
Banco de dados	Azure SQL Database, camada Básica	US$ 4,90
Código-fonte e integração contínua	GitHub	sem custo
Total		US$ 11,90

4 RESPONSABILIDADE SOCIAL E DIVERSIDADE

(Disciplina EAD: Relações Étnico-Raciais e Afrodescendência)

4.1 Combate à discriminação

O combate à discriminação foi tratado como conjunto de decisões técnicas verificáveis, e não como declaração de princípios. A **identificação emprega dado objetivo**: o sistema não coleta raça, religião, orientação sexual ou origem, o que reduz o risco de esses atributos servirem, ainda que involuntariamente, como critério de diferenciação — sem eliminá-lo, pois a discriminação também ocorre por fatores indiretos e por decisões humanas, que as medidas seguintes ajudam a conter e a rastrear. Os **critérios de avaliação são uniformes**: a correção compara a resposta ao gabarito, sem julgamento humano sujeito a vieses. A **segregação de privilégios** faz o escopo de visualização decorrer apenas da função exercida, com a regra aplicada no servidor. E a **rastreabilidade** do gatilho de auditoria da seção 8 registra toda alteração de perfil ou situação cadastral, tornando auditável eventual tratamento diferenciado.

4.2 Valorização da diversidade

A valorização da diversidade parte do reconhecimento de que os usuários acessam o sistema em condições materiais desiguais. Quanto aos **dispositivos**, o acesso predominante pelo celular — realidade de parcela expressiva dos estudantes brasileiros — motivou o aplicativo e a correção da responsividade da aplicação web. Quanto à **conexão**, o aplicativo mantém cópia local dos dados já carregados, consultáveis sem internet, com aviso de que podem estar desatualizados. Quanto ao **repertório tecnológico**, o assistente virtual aceita linguagem corrente e variações de escrita, sem distinguir acentos nem maiúsculas.

4.3 Recursos tecnológicos inclusivos

Cinco recursos foram implementados, disponíveis nas duas plataformas. O **modo de alto contraste** substitui a paleta por combinação de maior razão de contraste, com a preferência persistida entre sessões. As **escalas tipográficas** oferecem três níveis progressivos de ampliação, aplicados sem quebra do alinhamento. O **glossário em Língua Brasileira de Sinais** relaciona termos acadêmicos recorrentes à descrição do respectivo sinal, tendo sido ampliado dos três termos do PIM III para doze. A **rotulação para leitores de tela** atribuiu a todos os elementos interativos do aplicativo rótulos e papéis semânticos, permitindo navegação por tecnologia assistiva. E o **dimensionamento dos alvos de toque** observa o mínimo de 48 pontos, beneficiando usuários com limitação de coordenação motora.

4.4 Ações voltadas à acessibilidade

As ações adotadas orientaram-se pelas Diretrizes de Acessibilidade para Conteúdo Web (WCAG) e pela Lei Brasileira de Inclusão da Pessoa com Deficiência (Lei nº 13.146/2015), cujo artigo 63 estabelece a obrigatoriedade de acessibilidade em sítios eletrônicos. O quadro relaciona cada ação ao princípio correspondente:

Princípio WCAG	**Ação implementada**
Perceptível	Alto contraste; escalas de texto; rótulos textuais em ícones
Operável	Alvos de toque ampliados; encerramento de diálogos pela tecla Escape
Compreensível	Linguagem corrente; mensagens de erro explicativas; glossário em Libras
Robusto	Papéis semânticos declarados, compatíveis com leitores de tela

4.5 Tecnologia para a promoção da cidadania

A dimensão cidadã expressa-se em três frentes. A **transparência do desempenho** dá ao estudante acesso direto ao histórico de avaliações, com acertos e percentual de cada uma; a informação deixa de ser mediada exclusivamente pela instituição, condição para que o estudante acompanhe e questione a própria trajetória. A **autonomia no acesso**, viabilizada pela recuperação de senha por palavra-chave e pelo assistente virtual, reduz a dependência de intermediação administrativa em tarefas rotineiras — especialmente relevante para famílias com menor disponibilidade de deslocamento. E a **sustentabilidade ambiental** decorre da digitalização de avaliações e materiais: para 800 estudantes submetidos a avaliações periódicas, a economia projetada é da ordem de milhares de folhas por período letivo.

4.6 Farol de Evasão

A evasão é um dos problemas mais persistentes da educação brasileira e, em regra, só é percebida quando o estudante já deixou de frequentar. O PIM III previu, em caráter conceitual, a identificação de riscos de baixo rendimento a partir dos dados do sistema; o **Farol de Evasão** materializa essa proposta. Ele cruza cinco sinais já registrados pela plataforma e atribui a cada estudante uma pontuação de 0 a 100:

Sinal	**Regra**	**Pontos**
Desempenho	Aproveitamento médio inferior a 40% / a 60%	30 / 25
Inatividade	Sem acesso há 14 dias ou nunca acessou / há 7 dias	25 / 12
Leitura	Nenhum conteúdo lido nos últimos 14 dias	15
Pendências	Metade ou mais / ao menos uma atividade aberta há 7 dias sem entrega	20 / 10
Queda	Últimas três atividades 20 pontos abaixo das anteriores	10

A soma, que chega a no máximo 100 quando os cinco sinais disparam, define o nível — **risco alto** a partir de 60, **atenção** entre 30 e 59, **regular** abaixo de 30 —, e cada sinal disparado vem acompanhado de frase que o justifica, como "Rendimento caiu de 100% para 22,2% nas últimas 3 atividades". Optou-se deliberadamente por **regra transparente** em lugar de modelo estatístico: o docente precisa saber por que o estudante foi sinalizado para poder agir, e um classificador treinado exigiria histórico real de evasões, inexistente em instituição simulada.

Quatro decisões de projeto tratam a ferramenta com o cuidado que o tema exige. O farol **não é rótulo**: o estudante não tem acesso a ele, e a tela o apresenta como convite à conversa, não como diagnóstico. Há **minimização de dados** (Lei nº 13.709/2018, art. 6º, III): a secretaria vê a instituição, mas o docente vê apenas os estudantes das próprias turmas, e somente com dados das matérias dessas turmas. O nível **não é comunicado apenas pela cor**, em atendimento ao critério 1.4.1 das WCAG 2.1: cada nível tem também símbolo de formato distinto e rótulo escrito, distinguíveis por pessoas com daltonismo. E o cenário de demonstração usa **datas relativas** ao momento da carga, para que não envelheça até a apresentação.

A verificação partiu de seis perfis de estudante com trajetórias distintas, cuja pontuação foi calculada antes da execução; os seis resultados coincidiram com o previsto. O controle de acesso foi confirmado em produção: secretaria e docente recebem o farol, estudante recebe recusa 403 e requisição sem autenticação, recusa 401.

Figura 1 - Farol de Evasão no painel da secretaria



Fonte: Elaborado pelos Autores (2026)

5 DESENVOLVIMENTO DA SOLUÇÃO WEB

(Disciplina: Desenvolvimento Web com .NET)

5.1 Arquitetura da aplicação

A aplicação web foi construída sobre a plataforma **ASP.NET Core 10**, com a linguagem C#. A escolha decorreu de três fatores: a continuidade em relação ao PIM III, no qual o sistema já havia sido iniciado nessa tecnologia; a disponibilidade de um mapeador objeto-relacional maduro, o Entity Framework Core, que reduz o código repetitivo de acesso a dados; e o alinhamento com a disciplina de Programação Aplicada em .NET, cursada no mesmo semestre.

O modelo adotado é o de **aplicação monolítica com interface de programação REST**, no qual um único processo atende as requisições da interface e do aplicativo móvel. A opção pelo monólito foi deliberada: Newman (2019) adverte que a decomposição prematura em serviços introduz complexidade de rede, de implantação e de observabilidade que raramente se justifica em domínio único e equipe pequena. Para cerca de 800 usuários e seis desenvolvedores, o monólito bem estruturado em camadas oferece simplicidade operacional sem comprometer a organização interna do código, tema da seção 7.

O servidor embutido, Kestrel, atende as requisições e serve os arquivos estáticos da interface. Servir o front-end pela mesma aplicação eliminou uma classe inteira de problemas: como página e interface de programação compartilham origem, as chamadas usam caminho relativo e a aplicação funciona sem alteração em qualquer endereço.

Figura 2 - Tela de login da aplicação web



Fonte: Elaborado pelos Autores (2026)

5.2 Interface de programação REST

A interface expõe **41 endpoints** distribuídos em oito controladores, cada um responsável por um agregado do domínio, além da rota `/health`, mapeada diretamente na inicialização da aplicação:

Controlador	Rota base	Responsabilidade	Perfil exigido
AuthController	/api/auth	Cadastro, autenticação, logout e redefinição de senha	Público
UsuarioController	/api/usuarios	Gestão de contas e aprovações	Secretaria
MateriaController	/api/materias	Disciplinas e conteúdos didáticos	Professor / Aluno
TurmaController	/api/turmas	Turmas e matrículas	Professor / Secretaria
AtividadeController	/api/atividades	Avaliações e submissão de respostas	Professor / Aluno
RelatorioController	/api/relatorios	Relatórios, ranking e sessões	Conforme o dado
FarolController	/api/farol	Farol de Evasão (seção 4.6)	Secretaria / Professor
ChatbotController	/api/chatbot	Assistente virtual	Público
— (Program.cs)	/health	Verificação de disponibilidade	Público

Todas as respostas seguem envelope uniforme — `{ "sucesso": true, "dados": {…} }` em caso de êxito e `{ "sucesso": false, "mensagem": "…" }` em caso de falha —, padronização que permitiu ao front-end web e ao aplicativo móvel implementarem um único ponto de tratamento de erros, em vez de verificar cada chamada individualmente.

5.3 Autenticação

A autenticação emprega **JSON Web Token (JWT)**, padrão definido pela RFC 7519. O fluxo é o seguinte: o usuário envia CPF e senha; o servidor verifica a senha contra o hash armazenado; em caso de sucesso, emite um token assinado contendo as reivindicações de identidade, nome e perfil, com validade de oito horas.

A opção pelo token, em lugar de sessão em memória, atende a dois requisitos. A **ausência de estado no servidor** permite replicar a aplicação horizontalmente sem compartilhamento de sessão, sustentando a escalabilidade discutida na seção 9; e o mesmo mecanismo atende navegador e aplicativo sem adaptação, já que o token trafega em cabeçalho HTTP e não depende de cookies.

As senhas são armazenadas com **BCrypt**, algoritmo de hash deliberadamente lento e com sal embutido. Diferentemente de funções como SHA-256, projetadas para velocidade, o BCrypt impõe custo computacional a cada verificação, inviabilizando o teste massivo de combinações. A senha em texto claro nunca é gravada nem registrada em log.

5.4 Segurança

Além da autenticação, seis controles foram implementados. O **controle de acesso por perfis** é aplicado por atributos nos controladores e verificado no servidor a cada requisição: um estudante que alterasse a interface no navegador para exibir opções administrativas ainda receberia resposta 403, pois a decisão não depende do cliente. A **verificação de propriedade do dado** acrescenta a titularidade ao perfil — no relatório individual, o estudante obtém apenas o próprio desempenho, e consultar o de outro resulta em 403, impedindo que o simples conhecimento do CPF alheio dê acesso. A **proteção do gabarito** mantém as respostas corretas exclusivamente no banco: o objeto enviado ao estudante não inclui esse campo e a correção ocorre no servidor, o que impede obtê-lo por inspeção do tráfego. A **auditoria** registra alterações de perfil e situação cadastral por gatilho, conforme a seção 8.

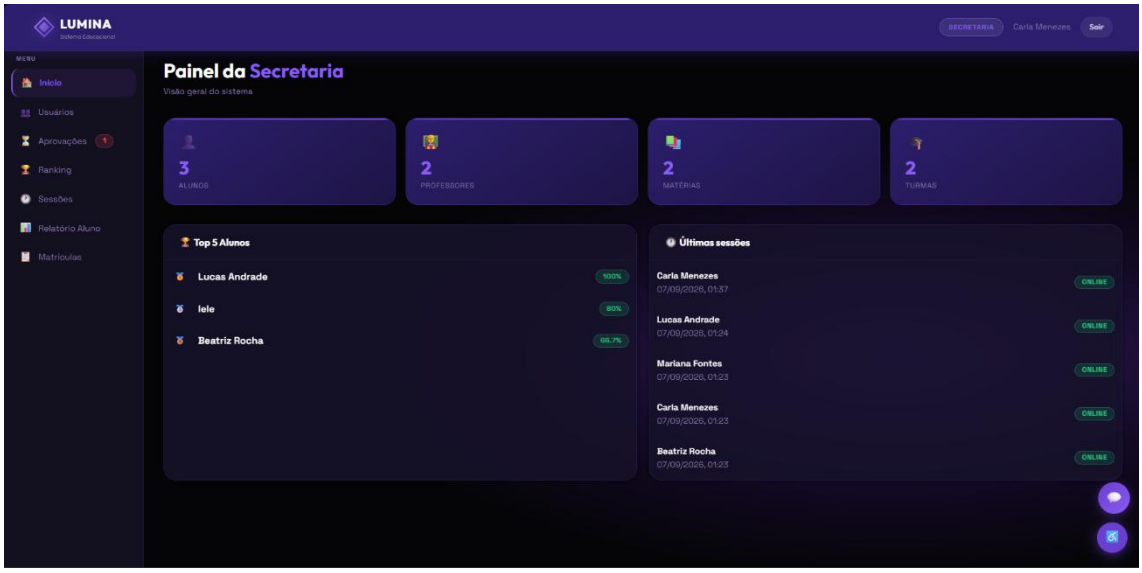
A **segregação de segredos** merece relato mais detido. Identificou-se que a chave de assinatura dos tokens e a senha administrativa estavam gravadas em arquivo de configuração versionado, em repositório público. A correção transferiu esses valores para o mecanismo de segredos do usuário, em desenvolvimento, e para variáveis de ambiente, em produção, além de **rotacionar a chave comprometida**. Acrescentou-se validação na inicialização: a aplicação recusa-se a subir se algum segredo estiver ausente ou se a chave tiver menos de 32 bytes, exigência do HMAC-SHA256. A exposição de segredos em repositório é uma das falhas mais comuns e mais facilmente exploráveis em projetos acadêmicos e profissionais.

Por fim, a **atualização de dependência vulnerável**: o pacote `Microsoft.OpenApi` 2.0.0, incluído por padrão pelo modelo de projeto, apresentava vulnerabilidade de severidade alta catalogada sob GHSA-v5pm-xwqc-g5wc. O pacote foi removido, e a verificação de pacotes vulneráveis passou a integrar o pipeline descrito na seção 9.

5.5 Estrutura administrativa

O perfil Secretaria dispõe de painel com indicadores consolidados de estudantes, docentes, disciplinas e turmas; gestão de identidades, incluindo cadastro, aprovação e revogação de contas; matrículas com validação automática de vagas; auditoria de sessões com data, hora e duração; relatórios de desempenho individual e ranking geral; e o Farol de Evasão (seção 4.6), exposto em `GET /api/farol` e restrito aos perfis Secretaria e Professor.

Figura 3 - Painel da Secretaria



Fonte: Elaborado pelos Autores (2026)

5.6 Assistente virtual

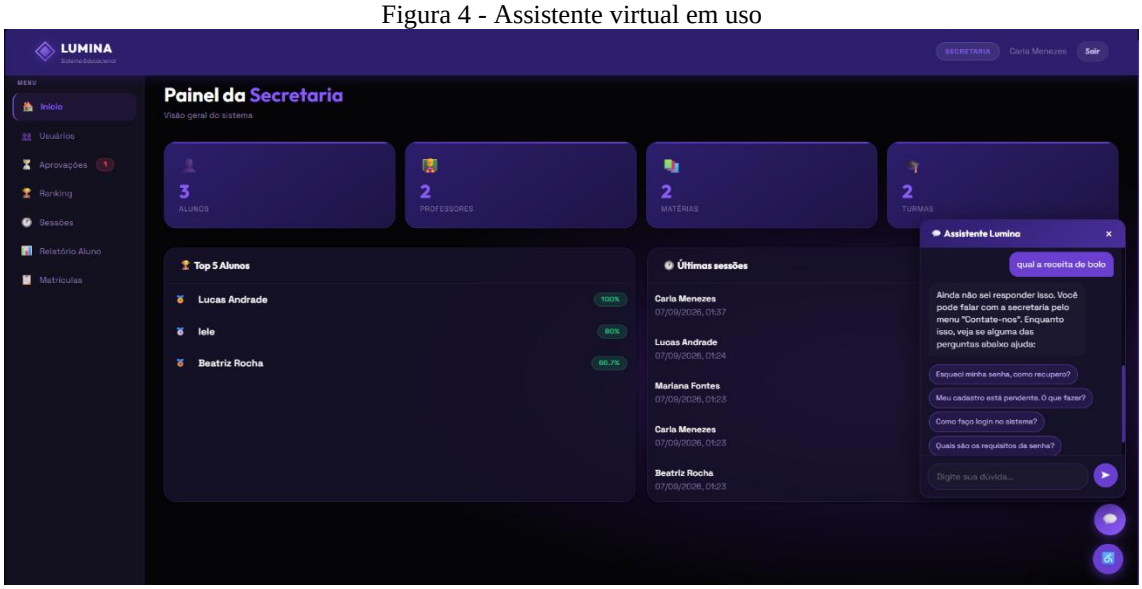
O assistente virtual, previsto no PIM III e não implementado naquela etapa, foi incorporado nesta fase. Opera sobre **base de conhecimento armazenada no próprio banco de dados**, com quinze perguntas categorizadas. O atendimento se dá por correspondência de palavras-chave: a pergunta é normalizada — convertida para minúsculas e despida de acentuação — e comparada com os termos cadastrados, somando três pontos por correspondência exata e um por parcial; a resposta de maior pontuação é devolvida, desde que ultrapasse um limiar mínimo de confiança.

A decisão de **não recorrer a serviços externos de inteligência artificial** merece registro, por contrariar a tendência atual. Sustentam-na três razões. A **proteção de dados**: perguntas de estudantes podem conter informações pessoais, e encaminhá-las a provedor externo implicaria transferência internacional de dados, hipótese que a Lei Geral de Proteção de Dados submete a requisitos específicos e que seria desproporcional ao benefício. A **previsibilidade**: modelos generativos produzem respostas plausíveis porém incorretas, e em contexto educacional uma orientação errada sobre prazo ou matrícula gera prejuízo concreto, enquanto a base curada garante que toda resposta tenha sido revisada. E o **custo e a autonomia**: serviços externos cobram por requisição e dependem de disponibilidade de terceiros, comprometendo o requisito de operação autônoma da instituição.

Quando não há correspondência confiável, o assistente **declara que não sabe** e oferece perguntas alternativas, encaminhando o usuário ao canal de contato da secretaria. O comportamento foi verificado com a pergunta "qual a receita de bolo", diante da qual o sistema

corretamente admitiu desconhecimento e apresentou quatro perguntas pertinentes ao domínio acadêmico, em vez de forçar uma resposta.

Figura 4 - Assistente virtual em uso



Fonte: Elaborado pelos Autores (2026)

5.7 Integração entre módulos

A integração entre a aplicação web, o aplicativo móvel e o banco ocorre exclusivamente pela interface REST. Não há acesso direto do cliente ao banco, o que concentra a validação das regras de negócio em um único ponto. Um efeito prático foi observado nos testes: uma matéria cadastrada pela interface web tornou-se imediatamente visível no aplicativo, sem sincronização adicional — ambos consultam a mesma fonte.

5.8 Correções decorrentes de teste em dispositivo real

O teste em telefone celular revelou dois defeitos ausentes no ambiente de desenvolvimento. O primeiro era um **endereço de interface fixo**: o front-end apontava para `http://localhost:5000/api`, e em um celular `localhost` refere-se ao próprio aparelho, não ao servidor — a porta, ademais, estava incorreta. A correção substituiu o endereço absoluto por caminho relativo (`/api`), fazendo as requisições seguirem para a mesma origem da página. O segundo era a **sobreposição de elementos na barra superior**: a marca institucional usava `position: absolute` com deslocamento central e cobria os atalhos de navegação em telas estreitas; uma consulta de mídia devolve o elemento ao fluxo normal abaixo de 640 pixels.

Ambos correspondem ao risco antecipado no PIM III, que registrara o "risco de quebra de componentes em telas de smartphones" para a persona Lucas. Nenhum dos dois apareceria em navegador de computador.

6 DESENVOLVIMENTO DA SOLUÇÃO MOBILE

(Disciplina: Desenvolvimento Mobile)

6.1 Tecnologia adotada e justificativa

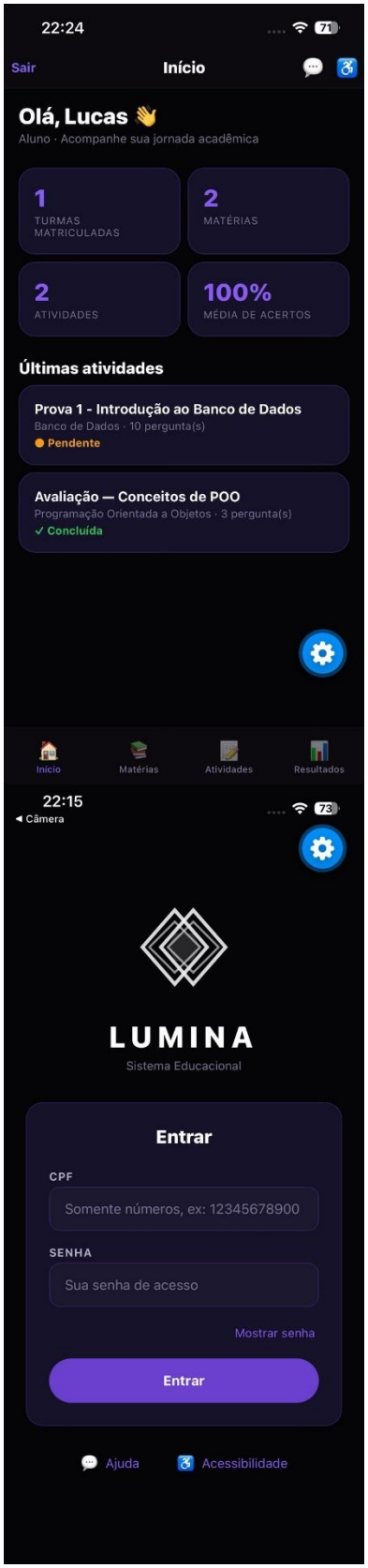
O aplicativo foi desenvolvido em **React Native**, com o ecossistema **Expo (SDK 57)**. A decisão exigiu análise, uma vez que a alternativa natural seria .NET MAUI, mantendo a uniformidade linguística com o restante do projeto.

O fator determinante foi prático: **compilar para iOS exige macOS e Xcode**. Nenhum integrante dispunha de computador Apple, e o dispositivo de teste era um iPhone; o MAUI permitiria apenas compilação para Android, restringindo a validação a emulador. O React Native com Expo contorna a limitação pelo aplicativo Expo Go, distribuído pela loja, que interpreta o código entregue pelo ambiente de desenvolvimento — viabilizando execução em iPhone real a partir de uma estação Windows. A contrapartida, a introdução de JavaScript, foi considerada aceitável porque a disciplina não impõe tecnologia específica e porque o alinhamento com .NET permanece na camada de serviço, integralmente C#.

6.2 Telas principais

O aplicativo compõe-se de sete telas: **Login**, com autenticação por CPF e senha; **Início**, com indicadores de engajamento e atividades recentes; **Matérias**, que apresenta disciplinas e conteúdos com registro automático de leitura; **Atividades**, separando pendentes de concluídas; **Responder Atividade**, com barra de progresso; **Meus Resultados**, com o histórico de desempenho; e **Assistente e Acessibilidade**, que reúne o apoio ao usuário e os recursos de inclusão.

Figura 5 - Tela de login do aplicativo; Painel inicial do aplicativo; Matérias e conteúdos; Execução de uma atividade


22:24
71
Sair
Início
Olá, Lucas 👋
Aluno · Acompanhe sua jornada acadêmica
1
TURMAS MATRICULADAS
2
MATÉRIAS
2
ATIVIDADES
100%
MÉDIA DE ACERTOS
Últimas atividades
Prova 1 - Introdução ao Banco de Dados
Banco de Dados · 10 pergunta(s)
Pendente
Avaliação — Conceitos de POO
Programação Orientada a Objetos · 3 pergunta(s)
✓ Concluída
Início
Matérias
Atividades
Resultados
22:15
◂ Câmera
73
LUMINA
Sistema Educacional
Entrar
CPF
Somente números, ex: 12345678900
SENHA
Sua senha de acesso
Mostrar senha
Entrar
Ajuda
Acessibilidade



Fonte: Elaborado pelos Autores (2026)

6.3 Fluxo de navegação

A navegação combina dois padrões. Um **navegador por abas** na parte inferior dá acesso às áreas principais — Início, Matérias, Atividades e Resultados —, enquanto uma **pilha de navegação** trata telas de contexto, como a execução de uma atividade e o assistente virtual.

A aba Resultados é exibida apenas ao perfil Aluno; docentes e secretaria acompanham desempenho pelos relatórios da aplicação web, mais adequados a dados agregados. O fluxo principal do estudante percorre a autenticação por CPF e senha, o painel inicial com indicadores e atividades pendentes, o acesso a Matérias com registro automático de leitura, a execução da atividade com retorno imediato do resultado e, por fim, a consulta ao histórico.

6.4 Autenticação e armazenamento seguro

O token recebido no login é gravado pelo **expo-secure-store**, que utiliza o Keychain no iOS e o Keystore no Android — serviços do sistema que cifram o conteúdo e o vinculam ao aplicativo. A alternativa comum, o `AsyncStorage`, grava **em texto claro**: em aparelho comprometido, o token poderia ser extraído e usado para personificar o usuário durante a validade da sessão. Ao iniciar, o aplicativo restaura a sessão e **verifica a expiração antes de considerá-la válida**, descartando tokens vencidos.

6.5 Sincronização de dados

O aplicativo mantém **cópia local dos dados consultados**, gravando cada leitura junto com o instante da coleta. Falhando a requisição por indisponibilidade de rede, a tela exibe a última cópia conhecida com aviso de possível desatualização. A estratégia é a de **cache com precedência da rede**: tenta-se sempre o dado atualizado, recorrendo à cópia local apenas em caso de falha. Erros de negócio — credencial inválida, permissão insuficiente — não acionam o cache, por serem respostas legítimas do servidor. No encerramento da sessão todo o cache é descartado, impedindo que dados de um usuário permaneçam acessíveis a outro no mesmo aparelho.

6.6 Integração com a interface de programação

O aplicativo consome os mesmos endpoints da aplicação web, compartilhando inclusive a base de usuários — as credenciais são idênticas nas duas plataformas.

A camada de acesso concentra três responsabilidades: montagem das requisições com injeção automática do token; normalização do envelope de resposta; e aplicação de tempo limite, evitando espera indefinida. Com a implantação da seção 9, o aplicativo passou a consumir a instância em nuvem e funciona em qualquer rede, inclusive dados móveis. O tempo

limite foi ampliado de quinze para sessenta segundos, pois a demonstração usa o plano gratuito, que hiberna (o Starter da seção 3.7 não) e leva cerca de 50 segundos para acordar; o aplicativo dispara a verificação de saúde ao ser aberto, e o servidor acorda enquanto o usuário digita as credenciais.

6.7 Acessibilidade no aplicativo

Os recursos descritos na seção 4 foram implementados também no aplicativo: alto contraste e três escalas tipográficas com preferências persistidas; glossário em Libras com doze termos; rótulos de acessibilidade e papéis semânticos em todos os elementos interativos, viabilizando leitor de tela; e alvos de toque de no mínimo 48 pontos.

Figura 6 - Recursos de acessibilidade no aplicativo



Fonte: Elaborado pelos Autores (2026)

A captura desta figura revelou um defeito que três revisões do código não haviam identificado. A paleta de alto contraste substitui a cor de destaque pelo amarelo, mas o texto

sobreposto a essas superfícies permanecia branco, produzindo razão de contraste de 1,07:1 quando a WCAG 2.1 exige 4,5:1 no nível AA. Cinco componentes eram afetados, entre eles o próprio interruptor de alto contraste: o texto que descreve o recurso tornava-se ilegível exatamente quando o recurso era acionado. A correção introduziu um token cromático específico para texto sobre superfícies de destaque, definido por paleta; em alto contraste, o preto sobre amarelo rende 19,6:1.

O episódio é instrutivo porque o defeito residia justamente no recurso destinado a mitigar barreiras visuais — declarar conformidade não a produz — e porque nenhuma leitura de código o revelaria: as duas cores estavam corretas isoladamente, e só a sobreposição, observada na tela do aparelho, o evidenciou.

6.8 Verificação

A verificação compreendeu duas frentes. A **compilação do pacote**, executada a cada alteração relevante, produziu sem erros o pacote de 2,2 MB destinado ao dispositivo, detectando importações inexistentes e erros de sintaxe antes que o código chegasse ao aparelho. A **execução em dispositivo real** foi feita em iPhone conectado à mesma rede local do servidor; registram-se as dificuldades enfrentadas, por constituírem aprendizado sobre desenvolvimento móvel:

O aparelho não alcançava o servidor porque o firewall bloqueava as portas em rede classificada como pública, resolvido por regra restrita à sub-rede local. O endereço deixou de responder após renovação por DHCP, corrigido pela atualização da configuração — dificuldades de rede local que a migração para a nuvem eliminou. O projeto foi recusado pelo aplicativo intermediário por divergência de SDK, resolvida pelo alinhamento à versão vigente, e novamente por falta de autenticação, já que o aplicativo estava vinculado a uma conta enquanto a ferramenta de linha de comando permanecia anônima.

A divergência de SDK ocorreu duas vezes, em sentidos opostos. O aplicativo intermediário usado para executar o projeto em dispositivo físico é distribuído pela loja da fabricante, que mantém apenas a versão mais recente. Quando o projeto foi criado, essa versão era anterior à dele, o que exigiu regredi-lo; semanas depois, o aplicativo atualizou-se automaticamente e passou a exigir a versão mais nova, tornando necessário desfazer a regressão. A dependência de um componente cuja versão não está sob controle da equipe é risco concreto de indisponibilidade em demonstrações agendadas, mitigado com a desativação da atualização automática no aparelho.

7 ARQUITETURA DE SOFTWARE

(Disciplina: Programação Aplicada em .NET)

7.1 Situação inicial e motivação

Ao término do PIM III, o sistema estava organizado em **projeto único**, com modelos, contexto de dados, serviços, controladores e ponto de entrada na raiz do repositório. A estrutura funcionava, mas não impunha fronteiras: nada impedia que um controlador acessasse o banco diretamente, contornando a camada de serviço.

Martin (2019) sustenta que o propósito da arquitetura é **postergar decisões e preservar opções**, mantendo as regras de negócio independentes de detalhes de infraestrutura. Sommerville (2019), por sua vez, associa a organização em camadas ao aumento da manutenibilidade, pela redução do acoplamento entre componentes.

7.2 Organização em camadas

A solução foi reorganizada em **quatro projetos**, com dependências explícitas:

```
SistemaEducacional.slnx
└── src/
    ├── SistemaEducacional.Domain/          (sem dependências)
    │   └── Entities/    — 12 entidades, uma por arquivo
    ├── SistemaEducacional.Infrastructure/  → Domain
    │   └── Data/        — contexto do Entity Framework
    ├── SistemaEducacional.Application/     → Domain, Infrastructure
    │   ├── DTOs/        — 31 objetos de transferência
    │   └── Services/    — 8 serviços de negócio
    └── SistemaEducacional.API/             → Application,
Infrastructure
        ├── Controllers/ — 8 controladores
        ├── Program.cs   — configuração e inicialização
        └── wwwroot/     — interface web
```

A **camada de Domínio** reúne as entidades do negócio e não referencia nenhum outro projeto nem bibliotecas de acesso a dados — ausência intencional, pois as regras do domínio não devem mudar porque o banco mudou. A **Infraestrutura** concentra o acesso a dados pelo contexto do Entity Framework, onde se configuram relacionamentos, chaves compostas e restrições de unicidade. A **Aplicação** contém os serviços que implementam os casos de uso, os objetos de transferência, as validações e a orquestração das operações. A **Apresentação** expõe a interface de programação e serve os arquivos da interface web, tratando exclusivamente de protocolo HTTP.

A separação entre entidade e objeto de transferência tem efeito direto sobre a segurança: a entidade `Usuario` possui o campo `SenhaHash`, e o objeto `UsuarioDto` não. Como os controladores devolvem apenas objetos de transferência, o hash **não pode ser exposto por descuido** — a omissão é estrutural, e não fruto de atenção do programador.

7.3 Orientação a objetos

Os princípios de orientação a objetos manifestam-se em quatro pontos. O **encapsulamento** faz os serviços exporem métodos que representam operações de negócio — `MatricularAsync`, `SubmeterAtividadeAsync` — mantendo privados os detalhes de execução, de modo que o controlador desconhece quantas consultas a operação envolve. A **abstração** faz os controladores dependerem da assinatura pública dos serviços, e não da implementação: substituir o Entity Framework por outro mecanismo exigiria alterar apenas a Infraestrutura. A **responsabilidade única** levou à divisão do arquivo de modelos original, com 316 linhas e 11 classes, em doze arquivos independentes. E a **injeção de dependência**, com os serviços recebendo suas dependências pelo construtor a partir do contêiner nativo, elimina o acoplamento a instâncias concretas e viabiliza substituí-las por implementações de teste.

7.4 Modularização

Cada camada é um projeto compilado independentemente, com dependências declaradas explicitamente. A consequência prática é que **violações arquiteturais tornam-se erros de compilação**: acessar o contexto de dados a partir do Domínio não compila, pois a referência não existe.

7.5 Verificação da refatoração

Uma refatoração dessa amplitude — 36 arquivos movidos ou criados — exige verificação, pois alterações estruturais introduzem defeitos silenciosos. Após a reorganização, verificou-se que a solução compila sem erros nem avisos, que a aplicação inicia e conecta ao banco, que a autenticação emite token válido, que endpoints restritos por perfil retornam corretamente os registros, que a interface web responde com HTTP 200 e que a documentação interativa permanece disponível.

A verificação confirmou que a reorganização preservou o comportamento observável do sistema — condição que define uma refatoração, por oposição a uma reescrita.

8 PROJETO DO BANCO DE DADOS

(Disciplina: Programação de Banco de Dados)

8.1 Justificativa do modelo relacional

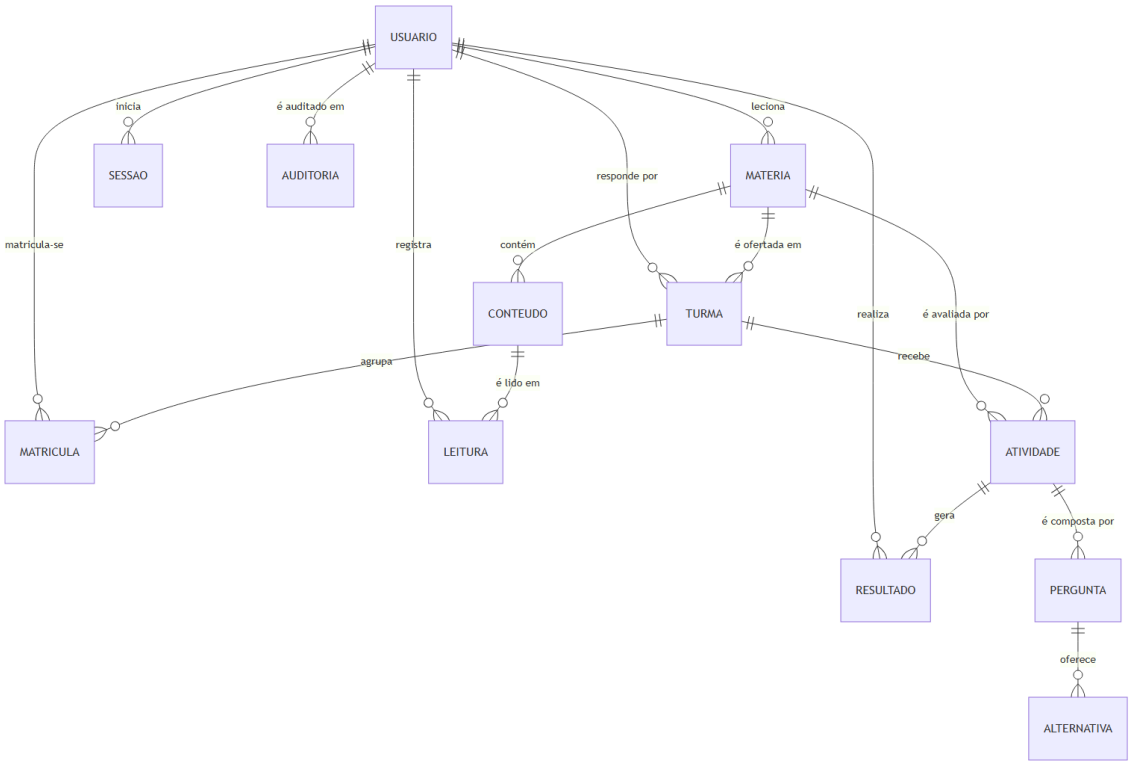
Adotou-se o modelo **relacional**, em Microsoft SQL Server, pela natureza dos dados: informações acadêmicas são altamente estruturadas e densamente relacionadas — estudantes vinculam-se a turmas, turmas a disciplinas, disciplinas a conteúdos e avaliações.

Elmasri e Navathe (2018) apontam como principal vantagem do modelo relacional a integridade referencial garantida pelo gerenciador: não é possível registrar resultado para estudante inexistente, qualquer que seja o código que tente. Bancos não relacionais trariam flexibilidade de esquema — pouco relevante para estrutura estável — ao custo de delegar a consistência à aplicação. Corrige-se, ainda, o PIM III, que mencionava banco não relacional na introdução e na conclusão: o sistema sempre utilizou SQL Server.

8.2 Modelo conceitual

O modelo conceitual, expresso pela abordagem Entidade-Relacionamento, abstrai detalhes de implementação e representa apenas as regras do negócio.

Figura 7 - Diagrama Entidade-Relacionamento



Fonte: Elaborado pelos Autores (2026)

Predominam relacionamentos de um para muitos — usuário e sessões, disciplina e conteúdos, avaliação e questões, questão e alternativas. Os de muitos para muitos são usuário e

turma, resolvido pela matrícula; usuário e atividade, pelo Resultado; e usuário e conteúdo, pelo registro de leitura.

8.3 Modelo lógico

O modelo lógico traduz o conceitual em estrutura de tabelas, definindo chaves primárias e propagando chaves estrangeiras. Os relacionamentos muitos-para-muitos foram normalizados por tabelas associativas com chave primária composta: `TurmaAlunos`, que representa a matrícula, e `LeituraConteudos`, que registra o acompanhamento de leitura.

O diagrama completo, por reunir as treze tabelas com seus atributos, chaves e relacionamentos, é apresentado em página própria no **Apêndice A**.

Normalização

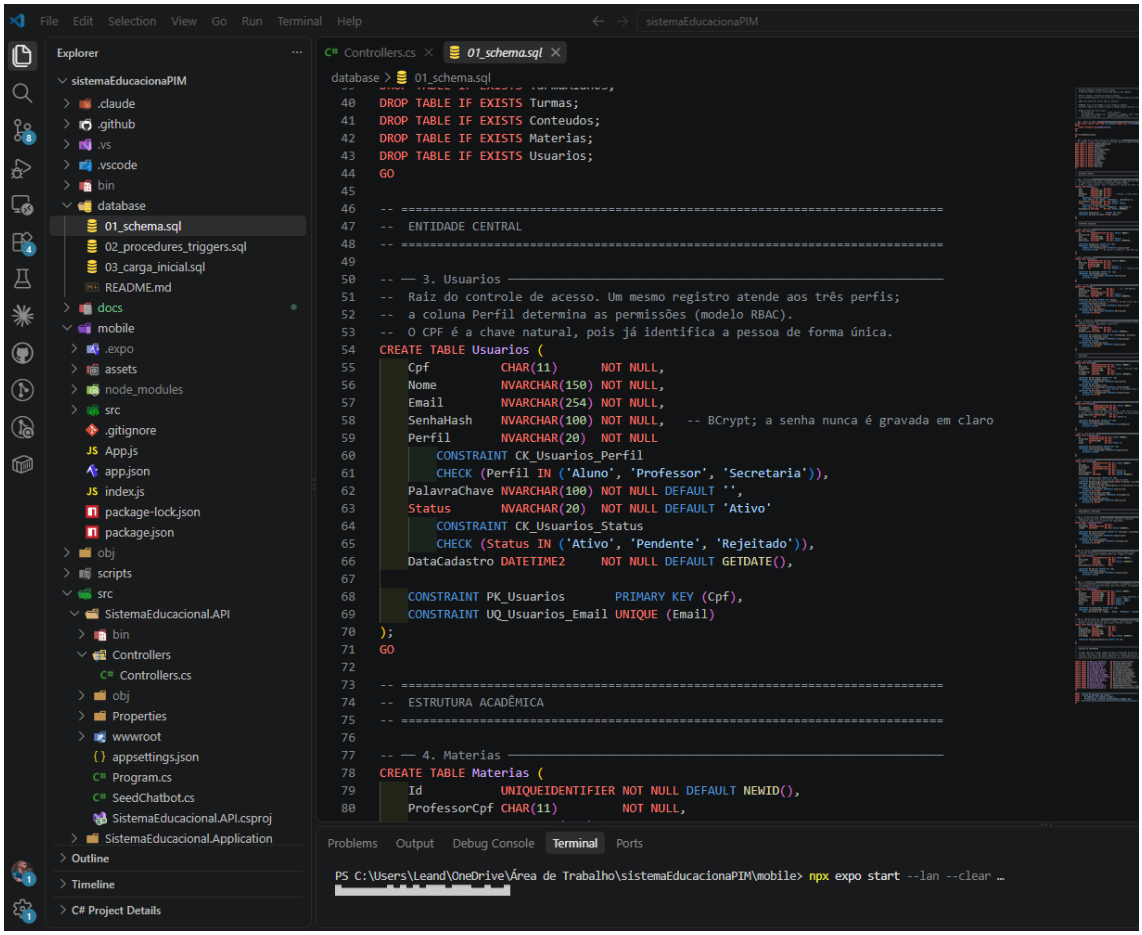
O modelo encontra-se na **Terceira Forma Normal**. Na **1FN**, todos os atributos são atômicos: as alternativas de uma questão ocupam tabela própria, e não uma lista delimitada por vírgulas em um único campo. Na **2FN**, nas tabelas de chave composta, os atributos dependem da chave integral. Na **3FN**, não há dependências transitivas — o nome do docente não se repete em `Materias`, onde se armazena o CPF, obtendo-se o nome por junção.

Há uma **exceção deliberada**: `Resultados` armazena `MateriaId`, obtenível por `Atividades`, porque relatórios por disciplina são a consulta mais frequente e a junção suprimida compensa o armazenamento.

8.4 Modelo físico

O modelo físico compreende **13 tabelas** e **15 índices** explícitos, definidos no script `01_schema.sql`.

Figura 8 - Estrutura física das tabelas



Fonte: Elaborado pelos Autores (2026)

As estratégias de exclusão foram diferenciadas conforme o significado do dado:

Estratégia	**Aplicação**	**Justificativa**
CASCADE	Conteúdos de uma matéria	O conteúdo não subsiste sem a disciplina
NO ACTION	Resultados de avaliações	Histórico acadêmico não deve ser apagado por efeito colateral
SET NULL	Turma de uma atividade	Removida a turma, a atividade torna-se geral em vez de ser destruída

8.5 Procedimentos armazenados

Foram implementados quatro procedimentos:

Procedimento	**Finalidade**
sp_RankingGeral	Classifica estudantes por aproveitamento, com filtro opcional por disciplina
sp_RelatorioAluno	Retorna resumo, histórico de avaliações e conteúdos lidos
sp_MatricularAluno	Efetiva matrícula com validação de perfil, situação e vagas
sp_DesempenhoTurma	Classifica a turma em faixas de rendimento

A **média é ponderada** pela razão entre acertos e questões, para que uma avaliação de duas questões não pese tanto quanto uma de vinte. E a matrícula usa os **bloqueios `UPDLOCK` e `HOLDLOCK`** ao contar vagas: sem eles, duas matrículas simultâneas leriam a mesma contagem e ambas seriam aceitas.

8.6 Gatilhos

Quatro gatilhos transpõem para o banco regras antes existentes apenas em documentação:

Gatilho	Regra aplicada
tr_TurmaAlunos_LimiteVagas	Impede ultrapassar 40 estudantes por turma (RN01)
tr_Resultados_Validar	Rejeita acertos negativos ou superiores ao total de questões
tr_Sessoes_CalcularDuracao	Calcula a duração da sessão no encerramento
tr_Usuarios_Auditoria	Registra alterações de perfil e situação (RN02)

A regra fica no banco para valer em **qualquer caminho de escrita**: restrição existente só no código é contornada por importação em massa ou acesso direto.

8.7 Verificação

Aplicados em banco criado do zero, os scripts geraram 13 tabelas, 15 índices, 4 procedimentos, 4 gatilhos e 1 visão, e o `tr_Resultados_Validar` rejeitou acertos iguais a 999 e a −5. No teste do limite de vagas, **tentou-se inserir 45 estudantes em uma turma**: o gatilho interrompeu a 41ª inserção, e a regra RN01, que no PIM III era linha de tabela, tornou-se restrição verificável.

Dois defeitos escaparam a essa verificação por não se manifestarem em consulta. O hash de senha das contas de demonstração provinha de exemplo da documentação do BCrypt, e **nenhuma conta autenticava** — corrigido gerando o hash com a biblioteca da aplicação e conferindo-o com `Verify()`. E os gatilhos, corretos isoladamente, **colidiam com o Entity Framework**: desde a versão 7, ele grava alterações com `UPDATE ... OUTPUT`, comando que o SQL Server recusa em tabela com gatilho. Redefinir senha, editar usuário e encerrar sessão falhavam, enquanto o login, que só insere, funcionava — o que ocultou o defeito até a produção. Declarar os gatilhos no mapeamento (`HasTrigger`) o corrigiu.

8.8 Script completo

O projeto do banco organiza-se em quatro scripts sequenciais: `01_schema.sql`, que cria o banco, as 13 tabelas e os 15 índices; `02_procedures_triggers.sql`, com os procedimentos, gatilhos, visão e tabela de auditoria; `03_carga_inicial.sql`, com os dados de demonstração; e `04_cenario_farol.sql`, com o cenário do Farol de Evasão.

Os scripts 2 a 4 são **idempotentes**: podem ser reaplicados sem erro. A íntegra dos scripts 1 e 2, que definem toda a estrutura do banco, está no **Apêndice B**; os scripts 3 e 4 trazem só dados de demonstração.

9 INFRAESTRUTURA EM NUVEM E DEVOPS

(Disciplina EAD: Cloud Computing e DevOps)

9.1 Arquitetura em nuvem

A solução está implantada e em operação em **https://lumina-sd21.onrender.com**, com o código publicado em https://github.com/lelezap-Dev/sistemaEducacionaPIM. A arquitetura encadeia três elementos:

```
Usuários → Render (Frankfurt)        — TLS; contêiner Docker da API e
da interface web
         → Azure SQL (Bélgica)       — SQL Server gerenciado, com
cópias de segurança
GitHub   → Render                    — publicação automática a cada
envio de código
```

A ausência de estado na aplicação, viabilizada pela autenticação por token descrita na seção 5, é o que permite replicar as instâncias sem compartilhamento de sessão.

9.2 Serviços utilizados

A aplicação executa no **Render**, em Plataforma como Serviço (PaaS), que dispensa equipe dedicada a servidores. O banco de dados é o **Azure SQL Database**, versão gerenciada do mesmo SQL Server usado em desenvolvimento — o que permitiu aplicar os quatro scripts da seção 8 **sem nenhuma alteração**, preservando procedimentos e gatilhos. O plano gratuito do Render oferece apenas PostgreSQL, o que exigiria converter todo o dialeto Transact-SQL.

A região foi condicionada pela conta estudantil: a brasileira foi bloqueada por política; a central norte-americana, por falta de capacidade; e a oferta gratuita do banco não existe na região belga, o que levou à camada Básica (seção 3.7). Como a aplicação consulta o banco várias vezes a cada ação, priorizou-se a proximidade entre os dois — banco na Bélgica, aplicação em Frankfurt, a cerca de 300 km —, e a consulta de verificação executa em **13 ms**.

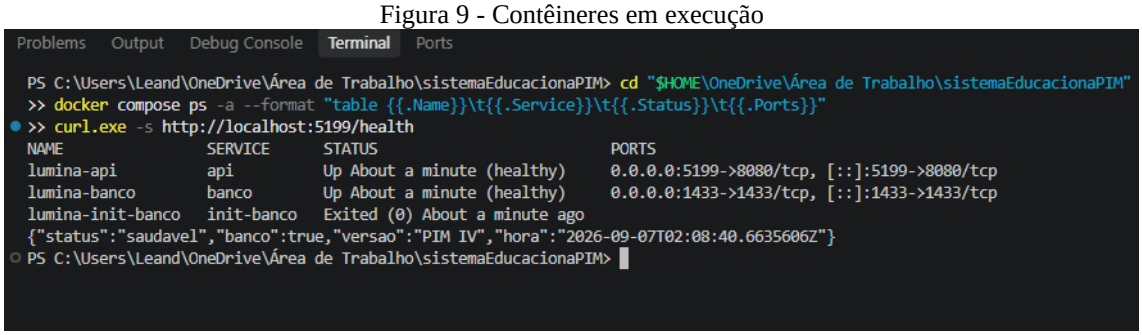
9.3 Contêineres

A aplicação foi conteinerizada com **Docker**, em construção de múltiplos estágios: a compilação usa a imagem do SDK do .NET (cerca de 800 MB), e a execução parte da imagem do runtime (cerca de 220 MB), recebendo apenas os binários publicados. A imagem final tem **376 MB** e não contém SDK, compiladores nem código-fonte, e a aplicação executa sob **usuário sem privilégios**.

O ambiente completo é descrito em `docker-compose.yml`, que orquestra três serviços: **banco**, o SQL Server, com verificação de disponibilidade que executa consulta real ao gerenciador; **init-banco**, que aplica os quatro scripts e encerra; e **api**, que só inicia após a

conclusão bem-sucedida do anterior. A sequência garante que a aplicação nunca encontre banco inexistente.

Figura 9 - Contêineres em execução



Fonte: Elaborado pelos Autores (2026)

9.4 Pipeline de integração e entrega contínuas

A **entrega contínua** é feita pelo Render, vinculado ao repositório: cada envio de código à ramificação do projeto dispara nova construção da imagem e nova implantação, e a versão só passa a receber tráfego depois de responder à verificação de saúde da seção 9.5 — uma versão incapaz de alcançar o banco não substitui a que está em operação.

A **integração contínua** é feita por pipeline em **GitHub Actions**, acionado a cada envio de código ou solicitação de incorporação, com três tarefas. A tarefa **API** restaura dependências, compila em modo Release com avisos tratados como erro e varre pacotes com vulnerabilidades conhecidas. A tarefa **Mobile** instala as dependências e compila o pacote. A tarefa **Docker** constrói a imagem, condicionada ao êxito da compilação da API.

A varredura automatiza a verificação que, feita à mão, identificou o pacote comprometido da seção 5.

9.5 Monitoramento

O monitoramento apoia-se no endpoint `/health`, que verifica não apenas a resposta da aplicação, mas também **o acesso ao banco de dados**, retornando código 503 quando este está inacessível. Em produção, o Render o consulta continuamente.

A implementação inicial verificava apenas a presença do runtime e reportaria saúde com a aplicação inoperante — indicador que nunca acusa falha produz falsa confiança. A versão corrigida foi verificada interrompendo o banco deliberadamente: o endpoint passou de 200 a 503 e voltou a 200.

9.6 Escalabilidade

Sem sessão em memória, qualquer instância atende qualquer requisição, e adicionar réplicas não exige coordenação. Em expansão significativa, as medidas seguintes seriam cache distribuído e réplicas de leitura no banco.

9.7 Segurança da infraestrutura

Seis medidas compõem a segurança do ambiente: os segredos permanecem fora do código, cadastrados como variáveis de ambiente no Render; o contêiner executa como usuário comum; a imagem final não contém SDK, compiladores nem código-fonte; a conexão com o banco é cifrada, com validação do certificado do servidor; as dependências passam por varredura automática no pipeline; e o **firewall do banco** recusa qualquer origem não autorizada, admitindo apenas o endereço da equipe e as faixas de saída do Render.

Ressalva: essas faixas são compartilhadas com outros clientes do provedor, e a proteção efetiva recai sobre a credencial do banco; isolamento completo exigiria endereço de saída dedicado, recurso pago.

A política de segredos foi posta à prova: uma senha de administrador escrita em script de apresentação chegou ao repositório público. Tratada como comprometida, foi substituída. Nas contas de demonstração, trocaram-se senha e palavra-chave de recuperação; as antigas passaram a ser recusadas.

9.8 Verificação do ambiente

O ambiente conteinerizado foi executado e verificado integralmente:

Verificação	**Resultado**
Construção da imagem	376 MB
Ordem de inicialização	banco → saudável → scripts → API
Aplicação dos scripts	13 tabelas, 4 procedimentos, 5 usuários
Autenticação	Bem-sucedida
Interface web	Resposta HTTP 200
Verificação de saúde sob falha	200 → 503 → 200
Scripts aplicados no Azure SQL	Sem alteração: 13 tabelas, 4 procedimentos, 4 gatilhos
Produção, verificação de saúde	200, banco acessível
Produção, autenticação e farol	Bem-sucedidos, com recusas 403 e 401 corretas

Nenhuma das configurações **funcionou na primeira execução**. Localmente, a imagem usava utilitário ausente, a verificação de saúde era inócua e a aplicação iniciava antes do banco. Na nuvem, a cadeia de conexão montada à mão estava malformada, e o firewall recusou a aplicação até a liberação de suas faixas — em ambos os casos, a aplicação recusou-se a iniciar com mensagem clara, em vez de falhar em silêncio.

10 GERENCIAMENTO ÁGIL DO PROJETO

(Disciplina: Gerenciamento de Projetos Ágil)

10.1 Metodologia adotada

O projeto foi conduzido segundo o framework **Scrum**, em continuidade ao adotado no PIM III. Schwaber e Sutherland (2020) definem o Scrum como estrutura leve para desenvolvimento de produtos complexos, apoiada em ciclos iterativos e inspeção frequente.

A escolha justifica-se pela natureza do trabalho: os requisitos, embora levantados no PIM III, sofreram ajustes durante a implementação — como a substituição da tecnologia móvel prevista, discutida na seção 6 —, e um método preditivo exigiria replanejamento formal a cada ajuste. O acompanhamento utilizou quadro **Kanban** na plataforma **Trello**, com as colunas Pendente, Em Andamento, Teste e Concluído.

10.2 Papéis

Ao **Product Owner** coube priorizar o backlog conforme os critérios de avaliação do PIM; ao **Scrum Master**, conduzir as reuniões e remover impedimentos; e ao **Time de Desenvolvimento**, a implementação, os testes e a documentação. Dada a dimensão da equipe — seis integrantes —, os papéis foram exercidos de forma compartilhada, com rodízio das responsabilidades de condução.

10.3 Product Backlog

O backlog do produto foi derivado das nove etapas exigidas pelo manual e priorizado segundo dois critérios: peso na avaliação e dependência técnica.

ID	História de usuário	Prior.	Est.
PB01	Como desenvolvedor, preciso de arquitetura em camadas	Alta	8
PB02	Como administrador, preciso de segredos fora do repositório	Alta	5
PB03	Como estudante, quero acessar o sistema pelo celular	Alta	21
PB04	Como estudante, quero responder atividades e ver o resultado	Alta	13
PB05	Como docente, quero acompanhar a turma por faixas	Média	8
PB06	Como secretaria, quero o limite de vagas respeitado	Alta	5
PB07	Como usuário, quero tirar dúvidas sem acionar a secretaria	Média	13

ID	História de usuário	Prior.	Est.
PB08	Como usuário com deficiência, quero autonomia de uso	Alta	8
PB09	Como equipe, queremos reproduzir o ambiente em qualquer máquina	Média	13
PB10	Como equipe, queremos detectar erros antes da entrega	Média	8
PB11	Como estudante, quero consultar dados sem conexão	Baixa	8
PB12	Como instituição, queremos rastrear alterações de privilégio	Média	5
PB13	Como docente, quero ser alertado sobre estudantes em risco de evasão	Alta	13
PB14	Como instituição, queremos o sistema acessível pela internet	Alta	8

As estimativas empregam a sequência de Fibonacci, em pontos de história, que representam esforço relativo e não duração absoluta.

10.4 Sprint Backlog

O desenvolvimento organizou-se em **cinco sprints de duas semanas**, com os itens do backlog distribuídos conforme a dependência técnica entre eles. Todos foram concluídos.

Sprint	Meta	Itens	Resultado
1 — Fundação e segurança	Base arquitetural sólida e eliminação de vulnerabilidades	PB01, PB02 e configuração do ambiente	Solução em quatro projetos, compilando sem avisos, com verificação funcional preservada
2 — Banco de dados	Transpor as regras de negócio para o banco	PB05, PB06, PB12 e consolidação dos scripts	4 procedimentos, 4 gatilhos e 1 visão, verificados em banco criado do zero
3 — Aplicação móvel	Disponibilizar acesso por dispositivo móvel	PB03, PB04, PB08, PB11	Aplicativo executado em dispositivo real
4 — Assistente e infraestrutura	Concluir funcionalidades pendentes e infraestrutura	PB07, PB09, PB10	Ambiente conteinerizado verificado em execução, com três defeitos corrigidos
5 — Diferencial e implantação	Entregar o diferencial e colocar o sistema em produção	PB13, PB14	Farol de Evasão verificado e sistema em operação na nuvem

O principal impedimento, na terceira sprint, foi a incompatibilidade de versão do SDK móvel (seção 6.8), que reapareceu semanas depois em sentido inverso.

10.5 Cronograma

Sprint	Período	Entrega
1	Semanas 1 e 2	Arquitetura em camadas e correção de segurança
2	Semanas 3 e 4	Banco de dados completo
3	Semanas 5 e 6	Aplicativo móvel
4	Semanas 7 e 8	Assistente virtual e infraestrutura

Sprint	Período	Entrega
5	Semanas 9 e 10	Farol de Evasão e implantação em nuvem
—	Semanas 11 e 12	Documentação e revisão final

Figura 10 - Quadro Kanban no Trello



Fonte: Elaborado pelos Autores (2026)

10.6 Definição de pronto

Um item só é considerado concluído quando satisfaz cinco critérios: funcionalidade implementada e compilando sem avisos; comportamento verificado por execução, e não apenas por compilação; código comentado onde a intenção não é evidente; alterações registradas no controle de versão com mensagem descritiva; e documentação correspondente atualizada.

O segundo critério foi decisivo. Diversos defeitos relatados neste trabalho — a verificação de saúde inócua, a ordem incorreta de inicialização, o endereço fixo no front-end, o hash de senha que não correspondia à senha anunciada e o texto ilegível sobre a superfície de alto contraste — passavam pela compilação sem indício de problema, e somente a execução os revelou. Os dois últimos não foram encontrados por teste planejado, mas durante a captura das imagens que ilustram este documento, ao usar o sistema como um usuário o usaria — o que sugere que a definição de pronto deveria exigir não apenas execução, mas execução observada por quem não escreveu o código.

11 CONCLUSÃO

O presente Projeto Integrado Multidisciplinar teve por objetivo desenvolver e implantar uma solução tecnológica corporativa integrada para apoiar a transformação digital da instituição Lumina, dando continuidade ao estudo conduzido no PIM III. Considera-se que o objetivo foi alcançado.

A aplicação web foi construída sobre a plataforma .NET, expondo interface de programação REST protegida por autenticação baseada em token e controle de acesso por perfis. O aplicativo móvel, desenvolvido em React Native, consome a mesma interface e compartilha a base de usuários, assegurando consistência entre as plataformas. A arquitetura foi reorganizada em quatro camadas com dependências explícitas, de modo que violações estruturais tornaram-se erros de compilação. O banco de dados, normalizado até a terceira forma normal, teve as regras de negócio transpostas para procedimentos e gatilhos. A infraestrutura foi conteinerizada, submetida a pipeline de integração contínua e implantada em nuvem, onde o sistema se encontra em operação com entrega contínua. Como diferencial, o Farol de Evasão converteu em funcionalidade verificável a análise de risco que o PIM III havia apenas proposto.

Retomando os objetivos específicos enunciados na introdução, verifica-se que: os dados foram centralizados em base única; a correção automática eliminou o intervalo entre a aplicação da avaliação e a divulgação do resultado; os indicadores de desempenho por faixa foram implementados, e o Farol de Evasão sinaliza precocemente os estudantes em risco; o assistente virtual oferece autoatendimento para dúvidas recorrentes; os recursos de acessibilidade estão presentes nas duas plataformas; e a digitalização das avaliações reduz o consumo de papel.

Do ponto de vista da formação, o trabalho evidenciou a articulação entre as disciplinas do semestre. A arquitetura em camadas, tratada em Programação Aplicada em .NET, condicionou a organização da aplicação web. A modelagem estudada em Programação de Banco de Dados materializou-se nos procedimentos e gatilhos. Os conceitos de Cloud Computing e DevOps orientaram a conteinerização. O gerenciamento ágil estruturou a condução. E os princípios discutidos na disciplina de Relações Étnico-Raciais e Afrodescendência traduziram-se em decisões técnicas verificáveis, e não em declarações genéricas.

Merece destaque particular o aprendizado quanto à **verificação por execução**. Diversos defeitos relatados ao longo deste documento — a verificação de saúde que nunca

acusaria falha, a ordem incorreta de inicialização dos contêineres, o endereço fixo que impedia o acesso por dispositivo móvel — atravessaram a compilação sem qualquer sinal. Foram revelados apenas quando o sistema foi efetivamente executado no ambiente de destino. A constatação de que **código que compila não é código que funciona** constitui, possivelmente, a lição mais transferível para a atuação profissional.

Reconhecem-se limitações. Os benefícios projetados na seção 3 não foram mensurados, dado tratar-se de instituição simulada; sua aferição exigiria implantação real e acompanhamento longitudinal. Os recursos de acessibilidade, embora orientados pelas diretrizes aplicáveis, não foram submetidos a teste com usuários com deficiência, procedimento indispensável em contexto profissional. E o sistema não foi submetido a teste de carga que permitisse afirmar seu comportamento sob volume elevado de acessos simultâneos.

Como desdobramentos futuros, sugerem-se: a implementação de testes automatizados, ausentes nesta etapa; a submissão dos recursos de acessibilidade a validação com usuários reais; a calibração dos pesos do Farol de Evasão com histórico real de evasões, quando houver; e a realização de teste de carga que fundamente as decisões de escalabilidade.

Conclui-se que a integração entre aplicações web e móveis, sustentada por arquitetura modular e banco de dados consistente, constitui caminho viável para a modernização tecnológica de instituições de ensino de porte médio. Mais do que a adoção de tecnologias específicas, o trabalho evidencia que a transformação digital efetiva depende de decisões arquiteturais que preservem a integridade da informação, da atenção à inclusão desde a concepção e da disciplina de verificar aquilo que se afirma ter construído.

REFERÊNCIAS

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

FOWLER, Martin. **Refatoração: aperfeiçoando o design de códigos existentes**.

2. ed. São Paulo: Novatec, 2020.

GAMMA, Erich et al. **Padrões de projeto: soluções reutilizáveis de software orientado a objetos**. Porto Alegre: Bookman, 2000.

MARTIN, Robert C. **Arquitetura limpa: o guia do artesão para estrutura e design de software**. Rio de Janeiro: Alta Books, 2019.

MICROSOFT. **Documentação do ASP.NET Core**. Disponível em: https://learn.microsoft.com/pt-br/aspnet/core/. Acesso em: 10 ago. 2026.

MICROSOFT. **Documentação do Entity Framework Core**. Disponível em: https://learn.microsoft.com/pt-br/ef/core/. Acesso em: 10 ago. 2026.

NEWMAN, Sam. **Construindo microsserviços**. 2. ed. São Paulo: Novatec, 2019.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

SCHWABER, Ken; SUTHERLAND, Jeff. **O guia do Scrum: o guia definitivo para o Scrum: as regras do jogo**. 2020. Disponível em: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-PortugueseBR.pdf. Acesso em: 10 ago. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

STALLINGS, William. **Criptografia e segurança de redes: princípios e práticas**. 6. ed. São Paulo: Pearson, 2015.

WORLD WIDE WEB CONSORTIUM. **Web Content Accessibility Guidelines (WCAG) 2.1**.

2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 10 ago. 2026.

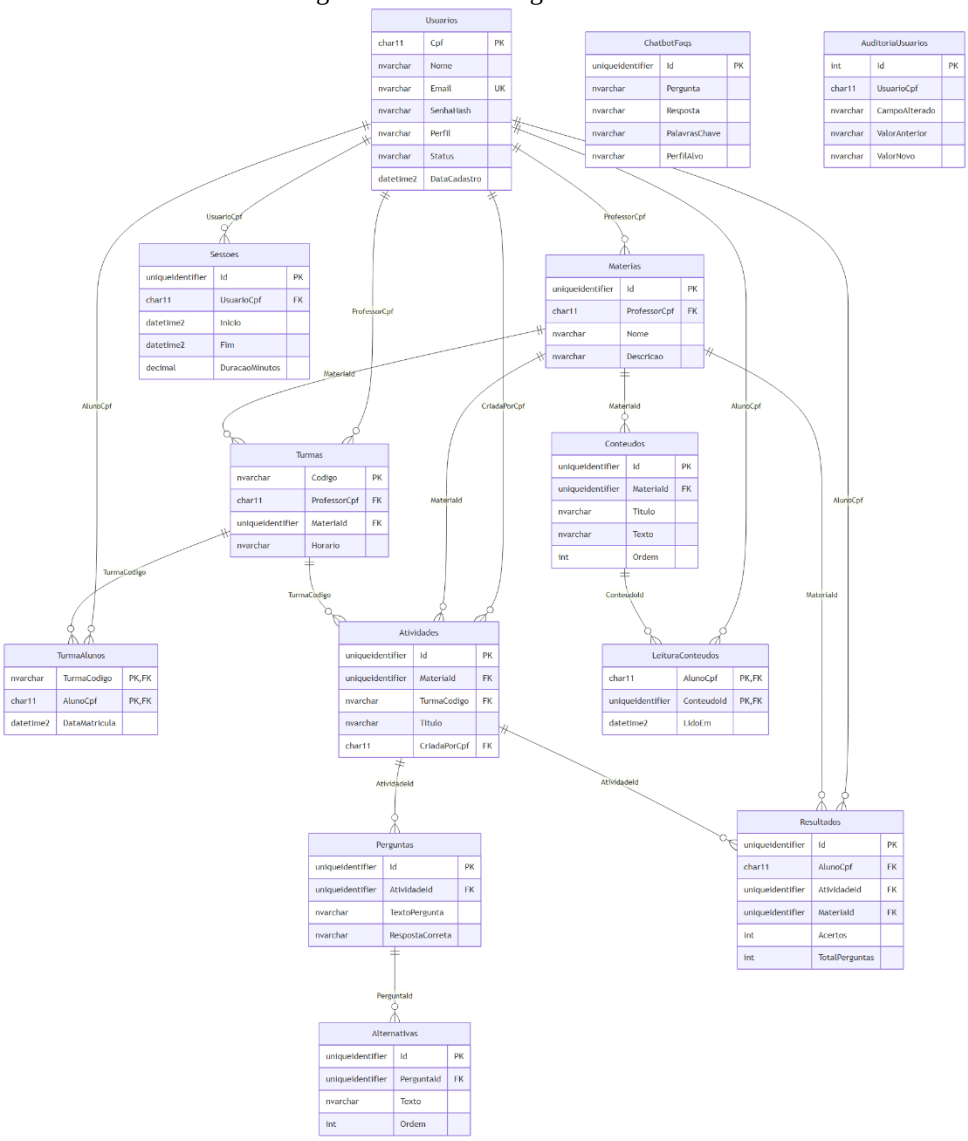
BRASIL. **Lei nº 13.146, de 6 de julho de 2015**. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência). Brasília, DF: Presidência da República, 2015.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

APÊNDICE A — MODELO LÓGICO DE DADOS

O diagrama reúne as treze tabelas do banco com seus atributos, tipos, chaves primárias e estrangeiras, conforme descrito na seção 8.3.

Figura 11 - Modelo lógico de dados



Fonte: Elaborado pelos Autores (2026)

APÊNDICE B — SCRIPT DO BANCO DE DADOS

Íntegra dos scripts que criam a estrutura do banco, conforme a seção 8.8, reproduzidos a partir dos arquivos da pasta `database` do repositório.

Script 1 — 01_schema.sql: banco, 13 tabelas e 15 índices.

```
-- ================================================================================
--  Sistema Acadêmico Colaborativo — Lumina
--  Script 01: Modelo Físico (criação do banco e das tabelas)
--
--  PIM IV — Etapa 7 (Projeto do Banco de Dados)
--  Universidade Paulista — CST em Análise e Desenvolvimento de Sistemas
--
--  SGBD: Microsoft SQL Server 2019 ou superior
--
--  ATENÇÃO: este script APAGA e recria todas as tabelas.
--  Execute-o apenas na primeira carga ou quando quiser reiniciar a base.
--
--  Ordem de execução dos scripts:
--     01_schema.sql               (este arquivo)
--     02_procedures_triggers.sql  (procedures, triggers, view e auditoria)
--     03_carga_inicial.sql        (dados de demonstração)
-- ================================================================================

-- ── 1. Banco de dados ──────────────────────────────────────────────────────────
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = N'SistemaEducacional')
BEGIN
    CREATE DATABASE SistemaEducacional;
END
GO

USE SistemaEducacional;
GO

-- ── 2. Remoção na ordem inversa das dependências ──────────────────────────────
-- As tabelas "filhas" saem primeiro para não violar as chaves estrangeiras.
DROP TABLE IF EXISTS AuditoriaUsuarios;
DROP TABLE IF EXISTS ChatbotFaqs;
DROP TABLE IF EXISTS Sessoes;
DROP TABLE IF EXISTS LeituraConteudos;
DROP TABLE IF EXISTS Resultados;
DROP TABLE IF EXISTS Alternativas;
DROP TABLE IF EXISTS Perguntas;
DROP TABLE IF EXISTS Atividades;
DROP TABLE IF EXISTS TurmaAlunos;
DROP TABLE IF EXISTS Turmas;
DROP TABLE IF EXISTS Conteudos;
DROP TABLE IF EXISTS Materias;
DROP TABLE IF EXISTS Usuarios;
GO

-- ================================================================================
--  ENTIDADE CENTRAL
-- ================================================================================

-- ── 3. Usuarios ────────────────────────────────────────────────────────────────
--  Raiz do controle de acesso. Um mesmo registro atende aos três perfis;
--  a coluna Perfil determina as permissões (modelo RBAC).
--  O CPF é a chave natural, pois já identifica a pessoa de forma única.
CREATE TABLE Usuarios (
    Cpf          CHAR(11)      NOT NULL,
    Nome         NVARCHAR(150) NOT NULL,
    Email        NVARCHAR(254) NOT NULL,
    SenhaHash    NVARCHAR(100) NOT NULL,   -- BCrypt; a senha nunca é gravada em
claro
    Perfil       NVARCHAR(20)  NOT NULL
```

```
        CONSTRAINT CK_Usuarios_Perfil
        CHECK (Perfil IN ('Aluno', 'Professor', 'Secretaria')),
    PalavraChave NVARCHAR(100) NOT NULL DEFAULT '',
    Status       NVARCHAR(20)  NOT NULL DEFAULT 'Ativo'
        CONSTRAINT CK_Usuarios_Status
        CHECK (Status IN ('Ativo', 'Pendente', 'Rejeitado')),
    DataCadastro DATETIME2     NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Usuarios       PRIMARY KEY (Cpf),
    CONSTRAINT UQ_Usuarios_Email UNIQUE (Email)
);
GO

-- ============================================================================
--  ESTRUTURA ACADÊMICA
-- ============================================================================

-- ── 4. Materias ──────────────────────────────────────────────────────────────
CREATE TABLE Materias (
    Id           UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    ProfessorCpf CHAR(11)         NOT NULL,
    Nome         NVARCHAR(200)    NOT NULL,
    Descricao    NVARCHAR(1000)   NOT NULL DEFAULT '',
    DataCriacao  DATETIME2        NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Materias PRIMARY KEY (Id),
    CONSTRAINT FK_Materias_Professor
        FOREIGN KEY (ProfessorCpf) REFERENCES Usuarios(Cpf)
        ON DELETE CASCADE   -- ao excluir o professor, suas matérias saem junto
);
GO

-- ── 5. Conteudos ─────────────────────────────────────────────────────────────
CREATE TABLE Conteudos (
    Id        UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    MateriaId UNIQUEIDENTIFIER NOT NULL,
    Titulo    NVARCHAR(300)    NOT NULL,
    Texto     NVARCHAR(MAX)    NOT NULL DEFAULT '',
    Ordem     INT              NOT NULL DEFAULT 0,   -- sequência de estudo

    CONSTRAINT PK_Conteudos PRIMARY KEY (Id),
    CONSTRAINT FK_Conteudos_Materia
        FOREIGN KEY (MateriaId) REFERENCES Materias(Id)
        ON DELETE CASCADE
);
GO

-- ── 6. Turmas ────────────────────────────────────────────────────────────────
CREATE TABLE Turmas (
    Codigo       NVARCHAR(50)     NOT NULL,   -- ex.: ADS-2026-N2
    ProfessorCpf CHAR(11)         NOT NULL,
    MateriaId    UNIQUEIDENTIFIER NOT NULL,
    Horario      NVARCHAR(100)    NOT NULL DEFAULT '',
    DataCriacao  DATETIME2        NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Turmas PRIMARY KEY (Codigo),
    -- NO ACTION evita que excluir um professor derrube turmas com alunos
    CONSTRAINT FK_Turmas_Professor
        FOREIGN KEY (ProfessorCpf) REFERENCES Usuarios(Cpf)
        ON DELETE NO ACTION,
    CONSTRAINT FK_Turmas_Materia
        FOREIGN KEY (MateriaId) REFERENCES Materias(Id)
        ON DELETE NO ACTION
);
GO

-- ── 7. TurmaAlunos ───────────────────────────────────────────────────────────
--  Tabela associativa que resolve o relacionamento N:N entre Turmas e
```

```
--  Usuarios (alunos). Representa a matrícula.
CREATE TABLE TurmaAlunos (
    TurmaCodigo   NVARCHAR(50) NOT NULL,
    AlunoCpf      CHAR(11)     NOT NULL,
    DataMatricula DATETIME2    NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_TurmaAlunos PRIMARY KEY (TurmaCodigo, AlunoCpf),
    CONSTRAINT FK_TurmaAlunos_Turma
        FOREIGN KEY (TurmaCodigo) REFERENCES Turmas(Codigo)
        ON DELETE CASCADE,
    CONSTRAINT FK_TurmaAlunos_Aluno
        FOREIGN KEY (AlunoCpf) REFERENCES Usuarios(Cpf)
        ON DELETE NO ACTION
);
GO

-- ============================================================================
--  AVALIAÇÃO
-- ============================================================================

-- ── 8. Atividades ──────────────────────────────────────────────────────────
CREATE TABLE Atividades (
    Id            UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    MateriaId     UNIQUEIDENTIFIER NOT NULL,
    TurmaCodigo   NVARCHAR(50)     NULL,   -- NULL = vale para todas as turmas
    Titulo        NVARCHAR(300)    NOT NULL,
    CriadaPorCpf  CHAR(11)         NOT NULL,
    CriadaEm      DATETIME2        NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Atividades PRIMARY KEY (Id),
    CONSTRAINT FK_Atividades_Materia
        FOREIGN KEY (MateriaId) REFERENCES Materias(Id)
        ON DELETE CASCADE,
    CONSTRAINT FK_Atividades_Turma
        FOREIGN KEY (TurmaCodigo) REFERENCES Turmas(Codigo)
        ON DELETE SET NULL,   -- turma removida: a atividade passa a ser geral
    CONSTRAINT FK_Atividades_Professor
        FOREIGN KEY (CriadaPorCpf) REFERENCES Usuarios(Cpf)
        ON DELETE NO ACTION
);
GO

-- ── 9. Perguntas ───────────────────────────────────────────────────────────
CREATE TABLE Perguntas (
    Id              UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    AtividadeId     UNIQUEIDENTIFIER NOT NULL,
    TextoPergunta   NVARCHAR(1000)   NOT NULL,
    -- A resposta correta fica somente no banco: a API nunca a envia ao
    -- aluno, o que impede que seja lida no navegador ou no aplicativo.
    RespostaCorreta NVARCHAR(500)    NOT NULL,
    Ordem           INT              NOT NULL DEFAULT 0,

    CONSTRAINT PK_Perguntas PRIMARY KEY (Id),
    CONSTRAINT FK_Perguntas_Atividade
        FOREIGN KEY (AtividadeId) REFERENCES Atividades(Id)
        ON DELETE CASCADE
);
GO

-- ── 10. Alternativas ───────────────────────────────────────────────────────
CREATE TABLE Alternativas (
    Id         UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    PerguntaId UNIQUEIDENTIFIER NOT NULL,
    Texto      NVARCHAR(500)    NOT NULL,
    Ordem      INT              NOT NULL DEFAULT 0,

    CONSTRAINT PK_Alternativas PRIMARY KEY (Id),
    CONSTRAINT FK_Alternativas_Pergunta
```

```
        FOREIGN KEY (PerguntaId) REFERENCES Perguntas(Id)
        ON DELETE CASCADE
);
GO

-- ── 11. Resultados ──────────────────────────────────────────────────────
CREATE TABLE Resultados (
    Id             UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    AlunoCpf       CHAR(11)         NOT NULL,
    AtividadeId    UNIQUEIDENTIFIER NOT NULL,
    MateriaId      UNIQUEIDENTIFIER NOT NULL,
    Acertos        INT              NOT NULL DEFAULT 0,
    TotalPerguntas INT              NOT NULL DEFAULT 0,
    RealizadoEm    DATETIME2        NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_Resultados PRIMARY KEY (Id),
    -- Garante um único envio por aluno em cada atividade
    CONSTRAINT UQ_Resultado_AlunoAtividade UNIQUE (AlunoCpf, AtividadeId),
    CONSTRAINT CK_Resultados_Acertos
        CHECK (Acertos >= 0 AND TotalPerguntas >= 0 AND Acertos <= TotalPerguntas),
    CONSTRAINT FK_Resultados_Aluno
        FOREIGN KEY (AlunoCpf) REFERENCES Usuarios(Cpf)
        ON DELETE NO ACTION,
    CONSTRAINT FK_Resultados_Atividade
        FOREIGN KEY (AtividadeId) REFERENCES Atividades(Id)
        ON DELETE NO ACTION,
    CONSTRAINT FK_Resultados_Materia
        FOREIGN KEY (MateriaId) REFERENCES Materias(Id)
        ON DELETE NO ACTION
);
GO

-- ============================================================================
--  ENGAJAMENTO E AUDITORIA
-- ============================================================================

-- ── 12. LeituraConteudos ────────────────────────────────────────────────
--  Registra que um aluno leu um conteúdo. Alimenta o indicador
--  "Conteúdos Lidos" e os relatórios de engajamento.
CREATE TABLE LeituraConteudos (
    AlunoCpf   CHAR(11)         NOT NULL,
    ConteudoId UNIQUEIDENTIFIER NOT NULL,
    LidoEm     DATETIME2        NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_LeituraConteudos PRIMARY KEY (AlunoCpf, ConteudoId),
    CONSTRAINT FK_Leitura_Aluno
        FOREIGN KEY (AlunoCpf) REFERENCES Usuarios(Cpf)
        ON DELETE CASCADE,
    CONSTRAINT FK_Leitura_Conteudo
        FOREIGN KEY (ConteudoId) REFERENCES Conteudos(Id)
        ON DELETE NO ACTION
);
GO

-- ── 13. Sessoes ─────────────────────────────────────────────────────────
--  Trilha de auditoria de acessos. Fim nulo indica sessão ativa.
--  A duração é calculada automaticamente por trigger no logout.
CREATE TABLE Sessoes (
    Id             UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    UsuarioCpf     CHAR(11)         NOT NULL,
    Inicio         DATETIME2        NOT NULL DEFAULT GETDATE(),
    Fim            DATETIME2        NULL,
    DuracaoMinutos DECIMAL(10,2)    NULL,

    CONSTRAINT PK_Sessoes PRIMARY KEY (Id),
    CONSTRAINT FK_Sessoes_Usuario
        FOREIGN KEY (UsuarioCpf) REFERENCES Usuarios(Cpf)
        ON DELETE CASCADE
```

```
);
GO

-- ── 14. ChatbotFaqs ──────────────────────────────────────────────────────
--  Base de conhecimento do assistente virtual. O atendimento se dá por
--  correspondência de palavras-chave sobre esta tabela, sem depender de
--  serviço externo de IA.
CREATE TABLE ChatbotFaqs (
    Id            UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
    Pergunta      NVARCHAR(300)    NOT NULL,
    Resposta      NVARCHAR(2000)   NOT NULL,
    PalavrasChave NVARCHAR(500)    NOT NULL,   -- termos separados por vírgula
    Categoria     NVARCHAR(50)     NOT NULL DEFAULT 'Geral',
    PerfilAlvo    NVARCHAR(20)     NOT NULL DEFAULT 'Todos',
    Ordem         INT              NOT NULL DEFAULT 0,
    Ativo         BIT              NOT NULL DEFAULT 1,

    CONSTRAINT PK_ChatbotFaqs PRIMARY KEY (Id),
    CONSTRAINT CK_ChatbotFaqs_Perfil
        CHECK (PerfilAlvo IN ('Todos', 'Aluno', 'Professor', 'Secretaria'))
);
GO

-- ── 15. AuditoriaUsuarios ────────────────────────────────────────────────
--  Histórico de alterações de perfil e status, preenchido por trigger.
--  Permite rastrear quem teve privilégios alterados e quando.
CREATE TABLE AuditoriaUsuarios (
    Id            INT IDENTITY(1,1) NOT NULL,
    UsuarioCpf    CHAR(11)          NOT NULL,
    CampoAlterado NVARCHAR(50)      NOT NULL,
    ValorAnterior NVARCHAR(100)     NULL,
    ValorNovo     NVARCHAR(100)     NULL,
    AlteradoEm    DATETIME2         NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_AuditoriaUsuarios PRIMARY KEY (Id)
);
GO

-- ============================================================================
--  ÍNDICES DE DESEMPENHO
--
--  Criados sobre as colunas usadas com maior frequência em filtros e
--  junções. Sem eles, o SQL Server percorreria a tabela inteira em
--  consultas como "matérias deste professor" ou "resultados deste aluno".
-- ============================================================================

CREATE INDEX IX_Materias_Professor    ON Materias(ProfessorCpf);
CREATE INDEX IX_Conteudos_Materia     ON Conteudos(MateriaId);
CREATE INDEX IX_Turmas_Professor      ON Turmas(ProfessorCpf);
CREATE INDEX IX_Turmas_Materia        ON Turmas(MateriaId);
CREATE INDEX IX_TurmaAlunos_Aluno     ON TurmaAlunos(AlunoCpf);
CREATE INDEX IX_Atividades_Materia    ON Atividades(MateriaId);
CREATE INDEX IX_Atividades_Turma      ON Atividades(TurmaCodigo);
CREATE INDEX IX_Perguntas_Atividade   ON Perguntas(AtividadeId);
CREATE INDEX IX_Alternativas_Pergunta ON Alternativas(PerguntaId);
CREATE INDEX IX_Resultados_Aluno      ON Resultados(AlunoCpf);
CREATE INDEX IX_Resultados_Materia    ON Resultados(MateriaId);
CREATE INDEX IX_Leitura_Aluno         ON LeituraConteudos(AlunoCpf);
CREATE INDEX IX_Sessoes_Usuario       ON Sessoes(UsuarioCpf);
CREATE INDEX IX_ChatbotFaqs_Perfil    ON ChatbotFaqs(PerfilAlvo, Ativo);
CREATE INDEX IX_Auditoria_Usuario     ON AuditoriaUsuarios(UsuarioCpf);
GO

PRINT '============================================================';
PRINT ' Script 01 aplicado com sucesso.';
PRINT '   13 tabelas e 15 indices criados.';
PRINT '   Proximo passo: executar 02_procedures_triggers.sql';
PRINT '============================================================';
```

```
GO
```

Script 2 — 02_procedures_triggers.sql: procedimentos, gatilhos, visão e tabela de auditoria.

```
-- ============================================================================
--  Sistema Acadêmico Colaborativo — Lumina
--  Script 02: Procedures, Triggers e Auditoria
--
--  PIM IV — Etapa 7 (Projeto do Banco de Dados)
--  Universidade Paulista — CST em Análise e Desenvolvimento de Sistemas
--
--  Pré-requisito: executar antes o script 01 (criação das tabelas).
--
--  Este script é IDEMPOTENTE: pode ser executado várias vezes sem erro,
--  pois todo objeto é removido antes de ser recriado.
-- ============================================================================

USE SistemaEducacional;
GO

-- ============================================================================
--  PARTE 1 — TABELA DE AUDITORIA
--  Registra alterações sensíveis de cadastro (perfil e status), atendendo à
--  rastreabilidade exigida pela regra de negócio RN02 do PIM III.
-- ============================================================================

IF OBJECT_ID('dbo.AuditoriaUsuarios', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.AuditoriaUsuarios (
        Id             INT IDENTITY(1,1) NOT NULL,
        UsuarioCpf     CHAR(11)          NOT NULL,
        CampoAlterado  NVARCHAR(50)      NOT NULL,
        ValorAnterior  NVARCHAR(100)     NULL,
        ValorNovo      NVARCHAR(100)     NULL,
        AlteradoEm     DATETIME2         NOT NULL DEFAULT GETDATE(),

        CONSTRAINT PK_AuditoriaUsuarios PRIMARY KEY (Id)
    );

    CREATE INDEX IX_AuditoriaUsuarios_Cpf  ON dbo.AuditoriaUsuarios(UsuarioCpf);
    CREATE INDEX IX_AuditoriaUsuarios_Data ON dbo.AuditoriaUsuarios(AlteradoEm
DESC);
END
GO

-- ============================================================================
--  PARTE 2 — PROCEDURES
-- ============================================================================

-- ----------------------------------------------------------------------------
--  sp_RankingGeral
--  Classifica os alunos pelo aproveitamento médio nas atividades.
--  Alimenta o módulo "Ranking" dos perfis Professor e Secretaria.
--
--  @TopN       quantidade de posições a retornar (padrão 10)
--  @MateriaId  filtra por matéria; NULL considera todas
-- ----------------------------------------------------------------------------
IF OBJECT_ID('dbo.sp_RankingGeral', 'P') IS NOT NULL
    DROP PROCEDURE dbo.sp_RankingGeral;
GO

CREATE PROCEDURE dbo.sp_RankingGeral
    @TopN      INT              = 10,
    @MateriaId UNIQUEIDENTIFIER = NULL
AS
BEGIN
    SET NOCOUNT ON;
```

```
    SELECT TOP (@TopN)
        Posicao         = ROW_NUMBER() OVER (
                              ORDER BY CAST(SUM(r.Acertos) AS DECIMAL(10,2))
                                     / NULLIF(SUM(r.TotalPerguntas), 0) DESC,
                                       COUNT(*) DESC
                          ),
        AlunoCpf        = u.Cpf,
        AlunoNome       = u.Nome,
        TotalAtividades = COUNT(*),
        TotalAcertos    = SUM(r.Acertos),
        TotalPerguntas  = SUM(r.TotalPerguntas),
        -- Média ponderada: acertos totais sobre perguntas totais.
        -- Evita distorção quando as atividades têm tamanhos diferentes.
        MediaPercentual = CAST(
                              100.0 * SUM(r.Acertos)
                              / NULLIF(SUM(r.TotalPerguntas), 0)
                            AS DECIMAL(5,2))
    FROM dbo.Resultados r
    INNER JOIN dbo.Usuarios u ON u.Cpf = r.AlunoCpf
    WHERE u.Perfil = 'Aluno'
      AND (@MateriaId IS NULL OR r.MateriaId = @MateriaId)
    GROUP BY u.Cpf, u.Nome
    ORDER BY MediaPercentual DESC, TotalAtividades DESC;
END
GO

-- ═══════════════════════════════════════════════════════════════════════
--  sp_RelatorioAluno
--  Consolida o desempenho individual de um aluno.
--  Retorna três conjuntos: resumo, histórico de atividades e leituras.
-- ═══════════════════════════════════════════════════════════════════════
IF OBJECT_ID('dbo.sp_RelatorioAluno', 'P') IS NOT NULL
    DROP PROCEDURE dbo.sp_RelatorioAluno;
GO

CREATE PROCEDURE dbo.sp_RelatorioAluno
    @AlunoCpf CHAR(11)
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM dbo.Usuarios WHERE Cpf = @AlunoCpf)
    BEGIN
        RAISERROR('Aluno não encontrado: %s', 16, 1, @AlunoCpf);
        RETURN;
    END

    -- (1) Resumo
    SELECT
        AlunoCpf          = u.Cpf,
        AlunoNome         = u.Nome,
        Email             = u.Email,
        TotalTurmas       = (SELECT COUNT(*) FROM dbo.TurmaAlunos     ta WHERE
ta.AlunoCpf = u.Cpf),
        TotalAtividades   = (SELECT COUNT(*) FROM dbo.Resultados       r WHERE
r.AlunoCpf  = u.Cpf),
        TotalConteudosLidos = (SELECT COUNT(*) FROM dbo.LeituraConteudos l WHERE
l.AlunoCpf = u.Cpf),
        MediaPercentual   = ISNULL((
                                SELECT CAST(100.0 * SUM(r.Acertos)
                                          / NULLIF(SUM(r.TotalPerguntas), 0) AS
DECIMAL(5,2))
                                FROM dbo.Resultados r
                                WHERE r.AlunoCpf = u.Cpf
                            ), 0)
    FROM dbo.Usuarios u
    WHERE u.Cpf = @AlunoCpf;
```

```
    -- (2) Histórico de atividades
    SELECT
        AtividadeTitulo = a.Titulo,
        MateriaNome     = m.Nome,
        Acertos         = r.Acertos,
        TotalPerguntas  = r.TotalPerguntas,
        Percentual      = CAST(100.0 * r.Acertos
                               / NULLIF(r.TotalPerguntas, 0) AS DECIMAL(5,2)),
        RealizadoEm     = r.RealizadoEm
    FROM dbo.Resultados r
    INNER JOIN dbo.Atividades a ON a.Id = r.AtividadeId
    INNER JOIN dbo.Materias   m ON m.Id = r.MateriaId
    WHERE r.AlunoCpf = @AlunoCpf
    ORDER BY r.RealizadoEm DESC;

    -- (3) Conteúdos lidos
    SELECT
        ConteudoTitulo = c.Titulo,
        MateriaNome    = m.Nome,
        LidoEm         = l.LidoEm
    FROM dbo.LeituraConteudos l
    INNER JOIN dbo.Conteudos c ON c.Id = l.ConteudoId
    INNER JOIN dbo.Materias  m ON m.Id = c.MateriaId
    WHERE l.AlunoCpf = @AlunoCpf
    ORDER BY l.LidoEm DESC;
END
GO

-- ═══════════════════════════════════════════════════════════════════════
--  sp_MatricularAluno
--  Efetiva a matrícula validando as regras de negócio no próprio banco,
--  de modo que a restrição valha para qualquer cliente (web, mobile ou
--  acesso direto ao SGBD).
--
--  Regras aplicadas (RN01 do PIM III):
--    - o CPF informado deve pertencer a um usuário de perfil Aluno
--    - o aluno precisa estar com cadastro Ativo
--    - a turma deve existir
--    - a turma não pode exceder o limite de vagas
--    - matrícula duplicada é rejeitada
-- ═══════════════════════════════════════════════════════════════════════
IF OBJECT_ID('dbo.sp_MatricularAluno', 'P') IS NOT NULL
    DROP PROCEDURE dbo.sp_MatricularAluno;
GO

CREATE PROCEDURE dbo.sp_MatricularAluno
    @AlunoCpf    CHAR(11),
    @TurmaCodigo NVARCHAR(50),
    @LimiteVagas INT = 40
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS (SELECT 1 FROM dbo.Usuarios
                       WHERE Cpf = @AlunoCpf AND Perfil = 'Aluno')
        BEGIN
            RAISERROR('CPF %s não corresponde a um aluno cadastrado.', 16, 1,
@AlunoCpf);
            ROLLBACK TRANSACTION; RETURN;
        END

        IF EXISTS (SELECT 1 FROM dbo.Usuarios
                   WHERE Cpf = @AlunoCpf AND Status <> 'Ativo')
        BEGIN
```

```
            RAISERROR('O aluno %s não está com cadastro ativo.', 16, 1, @AlunoCpf);
            ROLLBACK TRANSACTION; RETURN;
        END

        IF NOT EXISTS (SELECT 1 FROM dbo.Turmas WHERE Codigo = @TurmaCodigo)
        BEGIN
            RAISERROR('Turma %s não encontrada.', 16, 1, @TurmaCodigo);
            ROLLBACK TRANSACTION; RETURN;
        END

        IF EXISTS (SELECT 1 FROM dbo.TurmaAlunos
                   WHERE AlunoCpf = @AlunoCpf AND TurmaCodigo = @TurmaCodigo)
        BEGIN
            RAISERROR('O aluno já está matriculado nesta turma.', 16, 1);
            ROLLBACK TRANSACTION; RETURN;
        END

        DECLARE @Ocupacao INT;
        -- UPDLOCK/HOLDLOCK evitam que duas matrículas simultâneas leiam a
        -- mesma contagem e ultrapassem o limite de vagas juntas.
        SELECT @Ocupacao = COUNT(*)
        FROM dbo.TurmaAlunos WITH (UPDLOCK, HOLDLOCK)
        WHERE TurmaCodigo = @TurmaCodigo;

        IF @Ocupacao >= @LimiteVagas
        BEGIN
            RAISERROR('Turma %s lotada (%d de %d vagas ocupadas).',
                      16, 1, @TurmaCodigo, @Ocupacao, @LimiteVagas);
            ROLLBACK TRANSACTION; RETURN;
        END

        INSERT INTO dbo.TurmaAlunos (TurmaCodigo, AlunoCpf, DataMatricula)
        VALUES (@TurmaCodigo, @AlunoCpf, GETDATE());

        COMMIT TRANSACTION;

        SELECT
            Sucesso        = CAST(1 AS BIT),
            Mensagem       = 'Matrícula efetivada com sucesso.',
            VagasOcupadas  = @Ocupacao + 1,
            VagasRestantes = @LimiteVagas - (@Ocupacao + 1);
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
GO

-- ═══════════════════════════════════════════════════════════════════════
--  sp_DesempenhoTurma
--  Classifica os alunos de uma turma por faixa de rendimento.
--  Alimenta o módulo "Análise de Desempenho" do perfil Professor.
--
--  Faixas: Destaque (>= 80%), Em Progresso (60% a 79%), Em Risco (< 60%)
-- ═══════════════════════════════════════════════════════════════════════
IF OBJECT_ID('dbo.sp_DesempenhoTurma', 'P') IS NOT NULL
    DROP PROCEDURE dbo.sp_DesempenhoTurma;
GO

CREATE PROCEDURE dbo.sp_DesempenhoTurma
    @TurmaCodigo NVARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM dbo.Turmas WHERE Codigo = @TurmaCodigo)
    BEGIN
```

```
        RAISERROR('Turma %s não encontrada.', 16, 1, @TurmaCodigo);
        RETURN;
    END;

    -- CTE com a média de cada aluno matriculado na turma.
    -- O ponto e vírgula acima é exigido pelo T-SQL antes de um WITH.
    WITH MediasAlunos AS (
        SELECT
            u.Cpf,
            u.Nome,
            TotalAtividades = COUNT(r.Id),
            Media = ISNULL(CAST(100.0 * SUM(r.Acertos)
                                / NULLIF(SUM(r.TotalPerguntas), 0) AS DECIMAL(5,2)),
0)
        FROM dbo.TurmaAlunos ta
        INNER JOIN dbo.Usuarios   u ON u.Cpf = ta.AlunoCpf
        LEFT  JOIN dbo.Resultados r ON r.AlunoCpf = u.Cpf
        WHERE ta.TurmaCodigo = @TurmaCodigo
        GROUP BY u.Cpf, u.Nome
    )
    SELECT
        AlunoCpf        = Cpf,
        AlunoNome       = Nome,
        TotalAtividades,
        Media,
        Faixa = CASE
                    WHEN TotalAtividades = 0 THEN 'Sem atividades'
                    WHEN Media >= 80         THEN 'Destaque'
                    WHEN Media >= 60         THEN 'Em Progresso'
                    ELSE                          'Em Risco'
                END
    FROM MediasAlunos
    ORDER BY Media DESC, Nome;
END
GO

-- ============================================================================
--  PARTE 3 — TRIGGERS
-- ============================================================================

-- ----------------------------------------------------------------------------
--  tr_TurmaAlunos_LimiteVagas
--  Garante o limite de vagas mesmo em inserções feitas fora da procedure
--  (por exemplo, pelo Entity Framework ou por um INSERT manual).
--  É a última linha de defesa da regra RN01.
-- ----------------------------------------------------------------------------
IF OBJECT_ID('dbo.tr_TurmaAlunos_LimiteVagas', 'TR') IS NOT NULL
    DROP TRIGGER dbo.tr_TurmaAlunos_LimiteVagas;
GO

CREATE TRIGGER dbo.tr_TurmaAlunos_LimiteVagas
ON dbo.TurmaAlunos
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @LimiteVagas INT = 40;

    -- Verifica em conjunto: o INSERT pode trazer várias linhas de uma vez
    IF EXISTS (
        SELECT 1
        FROM dbo.TurmaAlunos ta
        WHERE ta.TurmaCodigo IN (SELECT DISTINCT TurmaCodigo FROM inserted)
        GROUP BY ta.TurmaCodigo
        HAVING COUNT(*) > @LimiteVagas
    )
    BEGIN
```

```
        RAISERROR('Limite de %d alunos por turma excedido. Matrícula cancelada.',
                  16, 1, @LimiteVagas);
        ROLLBACK TRANSACTION;
    END
END
GO

-- ═══════════════════════════════════════════════════════════════════════════
--  tr_Resultados_Validar
--  Impede o registro de resultados incoerentes: número de acertos maior que
--  o total de perguntas, ou valores negativos.
-- ═══════════════════════════════════════════════════════════════════════════
IF OBJECT_ID('dbo.tr_Resultados_Validar', 'TR') IS NOT NULL
    DROP TRIGGER dbo.tr_Resultados_Validar;
GO

CREATE TRIGGER dbo.tr_Resultados_Validar
ON dbo.Resultados
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM inserted
               WHERE Acertos < 0
                  OR TotalPerguntas < 0
                  OR Acertos > TotalPerguntas)
    BEGIN
        RAISERROR('Resultado inválido: acertos não podem ser negativos nem exceder
o total de perguntas.',
                  16, 1);
        ROLLBACK TRANSACTION;
    END
END
GO

-- ═══════════════════════════════════════════════════════════════════════════
--  tr_Sessoes_CalcularDuracao
--  Calcula automaticamente a duração da sessão quando o logout é registrado,
--  dispensando o cálculo na aplicação e mantendo o dado íntegro para a
--  auditoria da Secretaria.
-- ═══════════════════════════════════════════════════════════════════════════
IF OBJECT_ID('dbo.tr_Sessoes_CalcularDuracao', 'TR') IS NOT NULL
    DROP TRIGGER dbo.tr_Sessoes_CalcularDuracao;
GO

CREATE TRIGGER dbo.tr_Sessoes_CalcularDuracao
ON dbo.Sessoes
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Só age quando o campo Fim acabou de ser preenchido
    IF NOT UPDATE(Fim) RETURN;

    UPDATE s
    SET DuracaoMinutos = CAST(DATEDIFF(SECOND, s.Inicio, s.Fim) / 60.0 AS
DECIMAL(10,2))
    FROM dbo.Sessoes s
    INNER JOIN inserted i ON i.Id = s.Id
    INNER JOIN deleted  d ON d.Id = s.Id
    WHERE i.Fim IS NOT NULL
      AND d.Fim IS NULL;   -- evita recalcular sessões já encerradas
END
GO

-- ═══════════════════════════════════════════════════════════════════════════
```

```
--  tr_Usuarios_Auditoria
--  Registra alterações de Perfil e Status, permitindo rastrear quem teve
--  privilégios alterados e quando — exigência de segurança da Etapa 4.
-- ═══════════════════════════════════════════════════════════════════════
IF OBJECT_ID('dbo.tr_Usuarios_Auditoria', 'TR') IS NOT NULL
    DROP TRIGGER dbo.tr_Usuarios_Auditoria;
GO

CREATE TRIGGER dbo.tr_Usuarios_Auditoria
ON dbo.Usuarios
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Mudança de perfil (ex: Aluno promovido a Professor)
    INSERT INTO dbo.AuditoriaUsuarios (UsuarioCpf, CampoAlterado, ValorAnterior,
ValorNovo)
    SELECT i.Cpf, 'Perfil', d.Perfil, i.Perfil
    FROM inserted i
    INNER JOIN deleted d ON d.Cpf = i.Cpf
    WHERE ISNULL(i.Perfil, '') <> ISNULL(d.Perfil, '');

    -- Mudança de status (ex: Pendente -> Ativo na aprovação de cadastro)
    INSERT INTO dbo.AuditoriaUsuarios (UsuarioCpf, CampoAlterado, ValorAnterior,
ValorNovo)
    SELECT i.Cpf, 'Status', d.Status, i.Status
    FROM inserted i
    INNER JOIN deleted d ON d.Cpf = i.Cpf
    WHERE ISNULL(i.Status, '') <> ISNULL(d.Status, '');
END
GO

-- ═══════════════════════════════════════════════════════════════════════
--  PARTE 4 — VIEW DE APOIO
--  Consolida indicadores por matéria para os painéis gerenciais.
-- ═══════════════════════════════════════════════════════════════════════

IF OBJECT_ID('dbo.vw_IndicadoresMateria', 'V') IS NOT NULL
    DROP VIEW dbo.vw_IndicadoresMateria;
GO

CREATE VIEW dbo.vw_IndicadoresMateria
AS
SELECT
    MateriaId       = m.Id,
    MateriaNome     = m.Nome,
    ProfessorNome   = p.Nome,
    TotalConteudos  = (SELECT COUNT(*) FROM dbo.Conteudos  c WHERE c.MateriaId =
m.Id),
    TotalAtividades = (SELECT COUNT(*) FROM dbo.Atividades a WHERE a.MateriaId =
m.Id),
    TotalTurmas     = (SELECT COUNT(*) FROM dbo.Turmas     t WHERE t.MateriaId =
m.Id),
    MediaGeral      = ISNULL((
                          SELECT CAST(100.0 * SUM(r.Acertos)
                                    / NULLIF(SUM(r.TotalPerguntas), 0) AS
DECIMAL(5,2))
                          FROM dbo.Resultados r
                          WHERE r.MateriaId = m.Id
                      ), 0)
FROM dbo.Materias m
LEFT JOIN dbo.Usuarios p ON p.Cpf = m.ProfessorCpf;
GO

PRINT '============================================================';
PRINT ' Script 02 aplicado com sucesso.';
PRINT '   4 procedures  : sp_RankingGeral, sp_RelatorioAluno,';
```

```
PRINT '                    sp_MatricularAluno, sp_DesempenhoTurma';
PRINT '   4 triggers     : limite de vagas, validacao de resultados,';
PRINT '                    duracao de sessao, auditoria de usuarios';
PRINT '   1 view         : vw_IndicadoresMateria';
PRINT '   1 tabela       : AuditoriaUsuarios';
PRINT '============================================================';
GO
```